

ICPC Team Notebook

1	Template	1	4.2	Strongly Connected Components . . .	9	5.11	Maximum XOR Subset	24
1.1	Contest Template	1	4.3	LCA Binary Lifting	9	5.12	Number of Subset XOR Values	24
2	Data Structures	1	4.4	Bridges	10	5.13	XOR FWHT	25
2.1	Disjoint Set Union	1	4.5	Articulation Points	11	5.14	Discrete kth Roots mod Prime	25
2.2	Fenwick Tree	1	4.6	Dinic Max Flow	11	5.15	FFT	26
2.3	Sparse Table	2	4.7	Min Cost Flow	12	5.16	FFT Mod	27
2.4	Short Segment Tree	2	4.8	Kuhn Bipartite Matching	13	5.17	Fraction Finder	28
2.5	Priority Queue with Delete	3	4.9	Heavy Light Decomposition	13	5.18	Zeta and Mobius Transforms	28
2.6	Lazy Segment Tree	3	4.10	DSU on Tree Sack	14	6	Geometry	29
2.7	Line Container	4	4.11	Virtual Tree	15	6.1	Geometry Base	29
2.8	Implicit Treap	4	4.12	Kruskal Reconstruction Tree	16	6.2	Integer Geometry	30
2.9	DSU on Segment Tree	5	4.13	Global Min Cut	17	6.3	Convex Hull and Geometry Routines .	30
3	String	5	4.14	Maximum Clique	17	6.4	Point in Convex Polygon	31
3.1	KMP Prefix Function	5	4.15	Critical Cities Dominator	18	6.5	Closest Pair	32
3.2	Z Function	6	5	Math	20	6.6	Circle Tangents	32
3.3	Aho-Corasick	6	5.1	Number Theory Essentials	20	6.7	Minimum Enclosing Circle	32
3.4	Rolling Hash	7	5.2	Sieve	20	7	Dynamic Programming	33
3.5	Manacher Odd	7	5.3	CRT	20	7.1	Knuth Optimization	33
3.6	Minimum Rotation	7	5.4	Mod Inverse Table	21	7.2	Divide and Conquer DP	34
3.7	Suffix Array	8	5.5	Primitive Root	21	7.3	Divide and Conquer DP Notes	34
3.8	Kasai LCP	8	5.6	Miller Rabin and Pollard Rho	22	8	Misc	35
4	Graph	8	5.7	Gaussian Elimination mod p	22	8.1	Quick Select	35
4.1	Dijkstra	9	5.8	Catalan Numbers	23	8.2	mt19937 RNG	36
			5.9	Mobius Sieve	23			
			5.10	Mobius Inversion Notes	23			

8.3	mt19937_64 RNG	36	8.6	GNU Extensions	36	8.9	Debug Checklist	38
8.4	Random Hashing	36	8.7	GNU Extensions Notes	37			
8.5	Zobrist Hashing	36	8.8	Simulated Annealing Skeleton	38			

1 Template

1.1 Contest Template

Use: Default C++17 starting file.

Complexity: $O(1)$ setup.

Pitfalls: Uses `#define` `int long long`; remove multi-test loop if input has one case. Current return expression is intentional source text.

Assumptions/deps: Includes `bits/stdc++.h`, macros, pair printer, and `solve()`.

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define F first
5 #define S second
6 #define all(x) (x).begin(), (x).end()
7 #define log(x) cout << #x << " = " << x << " "
8 #define logl(x) cout << #x << " = " << x << endl
9 #define print(a) cout << #a << " = "; \
10 for(auto x: a) cout << x << " "; cout << endl;
11 template<typename T, typename U>
12 ostream &operator<<(ostream &os, const pair<T, U> &p)
13 {
14     return os << "(" << p.F << ", " << p.S << ")";
15 }
16
17 void solve()
18 {
19
20 }
21 int32_t main()
22 {
23     ios::sync_with_stdio(0); cin.tie(NULL);
24     int t; cin >> t; while(t--)
25         solve();
26     return 8 >> 13;
27 }
```

2 Data Structures

2.1 Disjoint Set Union

Use: Dynamic connectivity, Kruskal, components.

Complexity: Amortized $O(\alpha(n))$.

Pitfalls: Reinitialize per test case. `unite` returns false when already joined.

Assumptions/deps: 0-indexed; uses `vector`, `iota`.

```
1 // Disjoint Set Union.
2 // Complexity: amortized O(alpha(n)).
3 // Pitfalls: init per test case; unite returns false if already same set.
4 // Usage: DSU d(n); d.unite(u, v); d.same(u, v); d.size(u);
5 struct DSU {
6     vector<int> p, sz;
7     DSU(int n = 0) { init(n); }
8     void init(int n) {
9         p.resize(n);
10        iota(p.begin(), p.end(), 0);
11        sz.assign(n, 1);
12    }
13    int find(int x) {
14        return p[x] == x ? x : p[x] = find(p[x]);
15    }
16    bool unite(int a, int b) {
17        a = find(a), b = find(b);
18        if (a == b) return false;
19        if (sz[a] < sz[b]) swap(a, b);
20        p[b] = a;
21        sz[a] += sz[b];
22        return true;
23    }
24    bool same(int a, int b) {
25        return find(a) == find(b);
26    }
27    int size(int x) {
28        return sz[find(x)];
29    }
30 };
```

2.2 Fenwick Tree

Use: Prefix sums, range sums, kth by prefix.

Complexity: All operations $O(\log n)$.

Pitfalls: External indices are 0-based; `lower_bound` needs nonnegative frequencies.

Assumptions/deps: Uses `long long`; half-open query `[l,r)`.

```
1 // Fenwick tree for prefix sums.
```

```

2 // Complexity: add/sum/rangeSum/lower_bound are  $O(\log n)$ .
3 // Pitfalls: external indices are 0-based; lower_bound assumes all values nonnegative
4 // Usage: Fenwick fw(n); fw.add(i, x); fw.rangeSum(l, r); // [l,r)
5 struct Fenwick {
6     int n;
7     vector<long long> bit;
8     Fenwick(int n = 0) { init(n); }
9     void init(int n_) {
10         n = n_;
11         bit.assign(n + 1, 0);
12     }
13     void add(int idx, long long val) {
14         for (++idx; idx <= n; idx += idx & -idx) bit[idx] += val;
15     }
16     long long sumPrefix(int r) {
17         long long res = 0;
18         for (; r > 0; r -= r & -r) res += bit[r];
19         return res;
20     }
21     long long rangeSum(int l, int r) {
22         return sumPrefix(r) - sumPrefix(l);
23     }
24     int lower_bound(long long need) {
25         // first idx with sumPrefix(idx+1) >= need, or n if total < need
26         if (need <= 0) return 0;
27         int idx = 0;
28         for (int step = 1 << __lg(max(1, n)); step; step >>= 1) {
29             int next = idx + step;
30             if (next <= n && bit[next] < need) {
31                 idx = next;
32                 need -= bit[next];
33             }
34         }
35         return idx;
36     }
37 };

```

2.3 Sparse Table

Use: Static RMQ/min queries.

Complexity: Build $O(n \log n)$, query $O(1)$.

Pitfalls: Only idempotent ops for overlapping two-block query; range is $[l, r)$ and must be nonempty.

Assumptions/deps: Uses long long; adapt merge function for max/gcd.

```

1 // Sparse table for static range minimum query.
2 // Complexity: build  $O(n \log n)$ , query  $O(1)$ .

```

```

3 // Pitfalls: query range is [l,r); idempotent operations only unless adapted.
4 // Usage: SparseTable st(a); st.query(l, r);
5 struct SparseTable {
6     vector<vector<long long>> st;
7     vector<int> lg;
8     SparseTable() {}
9     SparseTable(const vector<long long>& a) { build(a); }
10    void build(const vector<long long>& a) {
11        int n = (int)a.size();
12        lg.assign(n + 1, 0);
13        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
14        st.assign(lg[n] + 1, vector<long long>(n));
15        st[0] = a;
16        for (int j = 1; j < (int)st.size(); j++)
17            for (int i = 0; i + (1 << j) <= n; i++)
18                st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
19    }
20    long long query(int l, int r) {
21        int j = lg[r - l];
22        return min(st[j][l], st[j][r - (1 << j)]);
23    }
24 };

```

2.4 Short Segment Tree

Use: Iterative point set and range sum.

Complexity: Build $O(n)$, update/query $O(\log n)$.

Pitfalls: Query is half-open $[l, r)$; N fixed at compile time.

Assumptions/deps: Fill $t[n+1]$ before build().

```

1 const int N = 1e5; // limit for array size
2 int n; // array size
3 long long t[2 * N];
4
5 void build() { // build the tree (first set positions n through 2n-1 as original
6     // array
7     for (int i = n - 1; i > 0; --i) t[i] = t[i << 1] + t[i << 1 | 1];
8 }
9
10 void modify(int p, long long value) { // set value at position p
11     for (t[p += n] = value; p > 1; p >>= 1) t[p >> 1] = t[p] + t[p ^ 1];
12 }
13
14 long long query(int l, int r) { // sum on interval [l, r)
15     long long res = 0;
16     for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
17         if (l & 1) res += t[l++];
18         if (r & 1) res += t[--r];
19     }
20 }

```

```

18 }
19 return res;
20 }

```

2.5 Priority Queue with Delete

Use: Lazy deletion for max heap.

Complexity: Amortized $O(\log n)$.

Pitfalls: `top()` assumes logical heap nonempty; delete only values inserted earlier.

Assumptions/deps: Uses two `priority_queue<int>`.

```

1 struct Heap {
2     // NOTE: top() assumes heap is nonempty and deletes were previously inserted.
3     priority_queue<int> qi, qo;
4     void ins(int v, int o) { (o>0?qi:qo).push(v); }
5     int top() {
6         while(qi.size() && qo.size() && qi.top()==qo.top()) qi.pop(), qo.pop();
7         return qi.top();
8     }
9 };

```

2.6 Lazy Segment Tree

Use: Range assignment and range sum.

Complexity: Build $O(n)$, operations $O(\log n)$.

Pitfalls: Inclusive bounds; empty ranges allowed. Change `apply/pull/neutral` for other operations.

Assumptions/deps: 0-indexed array; call recursive methods with `1,0,n-1`.

```

1 // Range assignment + range sum. 0-indexed, inclusive query/update bounds.
2 // Change apply/pull/neutral to adapt to other operations.
3 // Time: O(log N) per operation.
4 struct segTree {
5     int n;
6     vector<long long> sum, lazy;
7     vector<bool> has_lazy;
8
9     segTree(int nn = 0) { init(nn); }
10
11     void init(int nn) {
12         n = nn;
13         sum.assign(4 * max(1, n), 0);
14         lazy.assign(4 * max(1, n), 0);
15         has_lazy.assign(4 * max(1, n), false);

```

```

16 }
17
18 void build(const vector<long long>& a, int node, int l, int r) {
19     if (l == r) {
20         sum[node] = a[l];
21         return;
22     }
23     int m = (l + r) >> 1;
24     build(a, node << 1, l, m);
25     build(a, node << 1 | 1, m + 1, r);
26     sum[node] = sum[node << 1] + sum[node << 1 | 1];
27 }
28
29 void apply(int node, int l, int r, long long v) {
30     sum[node] = (r - l + 1) * v;
31     lazy[node] = v;
32     has_lazy[node] = true;
33 }
34
35 void push(int node, int l, int r) {
36     if (!has_lazy[node] || l == r) return;
37     int m = (l + r) >> 1;
38     apply(node << 1, l, m, lazy[node]);
39     apply(node << 1 | 1, m + 1, r, lazy[node]);
40     has_lazy[node] = false;
41 }
42
43 void assign_range(int node, int tl, int tr, int l, int r, long long v) {
44     if (l > r) return;
45     if (tl == l && tr == r) {
46         apply(node, tl, tr, v);
47         return;
48     }
49     push(node, tl, tr);
50     int tm = (tl + tr) >> 1;
51     assign_range(node << 1, tl, tm, l, min(r, tm), v);
52     assign_range(node << 1 | 1, tm + 1, tr, max(l, tm + 1), r, v);
53     sum[node] = sum[node << 1] + sum[node << 1 | 1];
54 }
55
56 long long range_query(int node, int tl, int tr, int l, int r) {
57     if (l > r) return 0;
58     if (tl == l && tr == r) return sum[node];
59     push(node, tl, tr);
60     int tm = (tl + tr) >> 1;
61     return range_query(node << 1, tl, tm, l, min(r, tm)) +
62         range_query(node << 1 | 1, tm + 1, tr, max(l, tm + 1), r);
63 }
64
65 long long point_query(int node, int tl, int tr, int pos) {
66     if (tl == tr) return sum[node];
67     push(node, tl, tr);

```

```

68     int tm = (tl + tr) >> 1;
69     if (pos <= tm) return point_query(node << 1, tl, tm, pos);
70     return point_query(node << 1 | 1, tm + 1, tr, pos);
71 }
72 };

```

2.7 Line Container

Use: Dynamic convex hull trick for max/min linear queries.

Complexity: Amortized $O(\log n)$ add/query.

Pitfalls: Default minimum=true; product $k*x+m$ can overflow. Duplicate slopes handled by code.

Assumptions/deps: Snippet defines `#define int long long`; requires `multiset`, `LLONG_MAX`.

```

1 #define int long long
2 struct Line {
3     // Requires notebook-wide "#define int long long" before this snippet.
4     // Without it, slopes/intercepts/intersections can overflow.
5     mutable int k, m, p;
6     bool operator<(const Line& o) const { return k < o.k; }
7     bool operator<(int x) const { return p < x; }
8 };
9
10 // queries for maximum
11 // if we want minimum then make minimum = true
12 bool minimum = true;
13 struct LineContainer : multiset<Line, less<>> {
14     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
15     static const int inf = LLONG_MAX;
16     int div(int a, int b) { // floored division
17         return a / b - ((a ^ b) < 0 && a % b); }
18     bool isect(iterator x, iterator y) {
19         if (y == end()) return x->p = inf, 0;
20         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
21         else x->p = div(y->m - x->m, x->k - y->k);
22         return x->p >= y->p;
23     }
24     // y = m*x+c; x will be the query parameter used later
25     void add(int m, int c) {
26         if (minimum){
27             m=-m;
28             c=-c;
29         }
30         auto z = insert({m, c, 0}), y = z++, x = y;
31         while (isect(y, z)) z = erase(z);
32         if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
33         while ((y = x) != begin() && (--x)->p >= y->p)

```

```

34         isect(x, erase(y));
35     }
36     int get(int x) {
37         assert(!empty());
38         auto l = *lower_bound(x);
39         if (minimum) return -(l.k * x + l.m);
40         return l.k * x + l.m;
41     }
42 };

```

2.8 Implicit Treap

Use: Sequence split/merge/reverse by position.

Complexity: Expected $O(\log n)$ per split/merge.

Pitfalls: Must allocate nodes and set `prior,value,cnt,rev,l,r`. Recursion depth randomized but possible.

Assumptions/deps: 0-indexed positions; call `push` before descending.

```

1 typedef struct item * pitem;
2 struct item {
3     int prior, value, cnt;
4     bool rev;
5     pitem l, r;
6 };
7
8 int cnt (pitem it) {
9     return it ? it->cnt : 0;
10 }
11
12 void upd_cnt (pitem it) {
13     if (it)
14         it->cnt = cnt(it->l) + cnt(it->r) + 1;
15 }
16
17 void push (pitem it) {
18     if (it && it->rev) {
19         it->rev = false;
20         swap (it->l, it->r);
21         if (it->l) it->l->rev ^= true;
22         if (it->r) it->r->rev ^= true;
23     }
24 }
25
26 void merge (pitem & t, pitem l, pitem r) {
27     push (l);
28     push (r);
29     if (!l || !r)
30         t = l ? l : r;

```

```

31     else if (l->prior > r->prior)
32         merge (l->r, l->r, r), t = l;
33     else
34         merge (r->l, l, r->l), t = r;
35     upd_cnt (t);
36 }
37
38 void split (pitem t, pitem & l, pitem & r, int key, int add = 0) {
39     if (!t)
40         return void( l = r = 0 );
41     push (t);
42     int cur_key = add + cnt(t->l);
43     if (key <= cur_key)
44         split (t->l, l, t->l, key, add), r = t;
45     else
46         split (t->r, t->r, r, key, add + 1 + cnt(t->l)), l = t;
47     upd_cnt (t);
48 }
49
50 void reverse (pitem t, int l, int r) {
51     pitem t1, t2, t3;
52     split (t, t1, t2, l);
53     split (t2, t2, t3, r-1+1);
54     t2->rev ^= true;
55     merge (t, t1, t2);
56     merge (t, t, t3);
57 }
58
59 void output (pitem t) {
60     if (!t) return;
61     push (t);
62     output (t->l);
63     printf ("%d ", t->value);
64     output (t->r);
65 }

```

2.9 DSU on Segment Tree

Use: Mergeable frequency/count segment trees, usually per DSU/tree node.

Complexity: Update/query $O(\log V)$; merge proportional to touched nodes.

Pitfalls: `tree[40*maxn]` and `tp` must be large/reset. Coordinate range `[tl,tr]` must match calls.

Assumptions/deps: Requires `maxn`; global node pool.

```

1 struct node {
2     int s, l, r;
3     node() : s(0), l(0), r(0) {}
4 } tree[40 * maxn];

```

```

5 int tp = 1;
6 void upd(int &u, int tl, int tr, int pos, int delta)
7 {
8     if(!u) u = tp++;
9     tree[u].s += delta;
10    if(tl < tr) {
11        int tm = (tl + tr) / 2;
12        if(pos <= tm) upd(tree[u].l, tl, tm, pos, delta);
13        else upd(tree[u].r, tm + 1, tr, pos, delta);
14    }
15 }
16 int query(int u, int tl, int tr, int l, int r)
17 {
18     if(l > r || u == 0) return 0;
19     if(l == tl && r == tr) return tree[u].s;
20     int tm = (tl + tr) / 2;
21     return query(tree[u].l, tl, tm, l, min(r, tm)) +
22            query(tree[u].r, tm + 1, tr, max(l, tm + 1), r);
23 }
24 int merge(int u, int v, int tl, int tr)
25 {
26     if(!u || !v) return u | v;
27     tree[u].s += tree[v].s;
28     if(tl < tr) {
29         int tm = (tl + tr) / 2;
30         tree[u].l = merge(tree[u].l, tree[v].l, tl, tm);
31         tree[u].r = merge(tree[u].r, tree[v].r, tm + 1, tr);
32     }
33     return u;
34 }

```

3 String

3.1 KMP Prefix Function

Use: Borders and single-pattern matching helper.

Complexity: $O(n)$.

Pitfalls: For matching, run on `pattern + sep + text`; choose `sep` absent from alphabet.

Assumptions/deps: 0-indexed prefix lengths.

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {

```

```

5     int j = pi[i-1];
6     while (j > 0 && s[i] != s[j])
7         j = pi[j-1];
8     if (s[i] == s[j])
9         j++;
10    pi[i] = j;
11 }
12 return pi;
13 }

```

3.2 Z Function

Use: Longest prefix match starting at each position.

Complexity: $O(n)$.

Pitfalls: For pattern matching, use separator not in alphabet. $z[0]=0$.

Assumptions/deps: 0-indexed string.

```

1 vector<int> z_function(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     int l = 0, r = 0;
5     for(int i = 1; i < n; i++) {
6         if(i < r) {
7             z[i] = min(r - i, z[i - l]);
8         }
9         while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
10            z[i]++;
11        }
12        if(i + z[i] > r) {
13            l = i;
14            r = i + z[i];
15        }
16    }
17    return z;
18 }

```

3.3 Aho-Corasick

Use: Count matches of many patterns in text.

Complexity: Build $O(\text{total length} \cdot A)$, scan $O(n)$.

Pitfalls: Only lowercase a..z; call build() after all patterns.

Assumptions/deps: Stores output counts, not pattern ids.

```

1 struct Aho {

```

```

2 // NOTE: only for lowercase 'a'..'z'. Other chars index out of bounds.
3 static const int A = 26;
4
5 struct Node {
6     int nxt[A];
7     int link = 0;
8     int out = 0;
9
10    Node() {
11        memset(nxt, -1, sizeof nxt);
12    }
13 };
14
15 vector<Node> t;
16
17 Aho() {
18     t.push_back(Node());
19 }
20
21 void add(const string& s) {
22     int v = 0;
23     for (char ch : s) {
24         int c = ch - 'a';
25         if (t[v].nxt[c] == -1) {
26             t[v].nxt[c] = t.size();
27             t.push_back(Node());
28         }
29         v = t[v].nxt[c];
30     }
31     t[v].out++;
32 }
33
34 void build() {
35     queue<int> q;
36
37     for (int c = 0; c < A; c++) {
38         int u = t[0].nxt[c];
39         if (u == -1) t[0].nxt[c] = 0;
40         else q.push(u);
41     }
42
43     while (!q.empty()) {
44         int v = q.front();
45         q.pop();
46
47         t[v].out += t[t[v].link].out;
48
49         for (int c = 0; c < A; c++) {
50             int u = t[v].nxt[c];
51             if (u == -1) {
52                 t[v].nxt[c] = t[t[v].link].nxt[c];
53             } else {

```



```

54         t[u].link = t[t[v].link].nxt[c];
55         q.push(u);
56     }
57 }
58 }
59 }
60
61 long long countMatches(const string& s) {
62     int v = 0;
63     long long ans = 0;
64     for (char ch : s) {
65         int c = ch - 'a';
66         v = t[v].nxt[c];
67         ans += t[v].out;
68     }
69     return ans;
70 }
71 };

```

3.4 Rolling Hash

Use: 64-bit polynomial hash for substrings.

Complexity: Build $O(n)$, query $O(1)$.

Pitfalls: Probabilistic collision; unsigned overflow is intentional.

Assumptions/deps: Half-open `get(l,r)`.

```

1 struct Hash {
2     static const uint64_t B = 1000003;
3     vector<uint64_t> h, p;
4
5     Hash(const string& s) {
6         int n = s.size();
7         h.assign(n + 1, 0);
8         p.assign(n + 1, 1);
9         for (int i = 0; i < n; i++) {
10             h[i + 1] = h[i] * B + (unsigned char)s[i] + 1;
11             p[i + 1] = p[i] * B;
12         }
13     }
14
15     // hash of s[l..r)
16     uint64_t get(int l, int r) {
17         return h[r] - h[l] * p[r - l];
18     }
19 };

```

3.5 Manacher Odd

Use: Odd palindrome radii.

Complexity: $O(n)$.

Pitfalls: This returns odd radii only; sentinels `$` and `^` must not conflict if adapted.

Assumptions/deps: For even palindromes use separate transform/template.

```

1 vector<int> manacher_odd(string s) {
2     int n = s.size();
3     s = "$" + s + "^";
4     vector<int> p(n + 2);
5     int l = 0, r = 1;
6     for(int i = 1; i <= n; i++) {
7         p[i] = min(r - i, p[l + (r - i)]);
8         while(s[i - p[i]] == s[i + p[i]]) {
9             p[i]++;
10        }
11        if(i + p[i] > r) {
12            l = i - p[i], r = i + p[i];
13        }
14    }
15    return vector<int>(begin(p) + 1, end(p) - 1);
16 }

```

3.6 Minimum Rotation

Use: Lexicographically minimum cyclic rotation index.

Complexity: $O(n)$.

Pitfalls: For periodic strings returns one minimum index. Empty string not handled.

Assumptions/deps: 0-indexed index in original string.

```

1 int min_rotation(string s) {
2     s += s;
3     int n = s.size(), i = 0, ans = 0;
4
5     while (i < n / 2) {
6         ans = i;
7         int j = i + 1, k = i;
8
9         while (j < n && s[k] <= s[j]) {
10             if (s[k] < s[j]) k = i;
11             else k++;
12             j++;
13         }
14
15         while (i <= k) i += j - k;
16     }

```

```

17     return ans;
18 }
19 }

```

3.7 Suffix Array

Use: Sort cyclic shifts; use sentinel for suffix array.

Complexity: $O(n \log n)$.

Pitfalls: Append unique smallest sentinel and remove it for real suffix array. Empty handled.

Assumptions/deps: Alphabet bytes 0..255; returns cyclic shift order.

```

1  // NOTE: cyclic shifts only. Append unique sentinel and remove it for suffix array.
2  vector<int> sort_cyclic_shifts(string const& s) {
3      int n = s.size();
4      if (n == 0) return {};
5      const int alphabet = 256;
6      auto code = [&](int i) { return (unsigned char)s[i]; };
7      vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
8      for (int i = 0; i < n; i++)
9          cnt[code(i)]++;
10     for (int i = 1; i < alphabet; i++)
11         cnt[i] += cnt[i-1];
12     for (int i = 0; i < n; i++)
13         p[--cnt[code(i)]] = i;
14     c[p[0]] = 0;
15     int classes = 1;
16     for (int i = 1; i < n; i++) {
17         if (code(p[i]) != code(p[i-1]))
18             classes++;
19         c[p[i]] = classes - 1;
20     }
21     vector<int> pn(n), cn(n);
22     for (int h = 0; (1 << h) < n; ++h) {
23         for (int i = 0; i < n; i++) {
24             pn[i] = p[i] - (1 << h);
25             if (pn[i] < 0)
26                 pn[i] += n;
27         }
28         fill(cnt.begin(), cnt.begin() + classes, 0);
29         for (int i = 0; i < n; i++)
30             cnt[c[pn[i]]]++;
31         for (int i = 1; i < classes; i++)
32             cnt[i] += cnt[i-1];
33         for (int i = n-1; i >= 0; i--)
34             p[--cnt[c[pn[i]]]] = pn[i];
35         cn[p[0]] = 0;
36         classes = 1;

```

```

37     for (int i = 1; i < n; i++) {
38         pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
39         pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
40         if (cur != prev)
41             ++classes;
42         cn[p[i]] = classes - 1;
43     }
44     c.swap(cn);
45 }
46 return p;
47 }

```

3.8 Kasai LCP

Use: LCP array from string and suffix array.

Complexity: $O(n)$.

Pitfalls: Requires valid suffix array for same string. Output size is $n-1$.

Assumptions/deps: Pairs with suffix array snippet.

```

1  // Kasai LCP array from string s and suffix array sa.
2  // lcp[i] = LCP(sa[i], sa[i+1]), size max(0, n-1).
3  // Time: O(n). Source: standard Kasai algorithm (cp-algorithms/USACO Guide).
4  vector<int> lcp_array(const string& s, const vector<int>& sa) {
5      int n = (int)s.size();
6      if (n == 0) return {};
7      vector<int> rank(n), lcp(max(0, n - 1));
8      for (int i = 0; i < n; i++) rank[sa[i]] = i;
9      int h = 0;
10     for (int i = 0; i < n; i++) {
11         int r = rank[i];
12         if (r == n - 1) {
13             h = 0;
14             continue;
15         }
16         int j = sa[r + 1];
17         while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
18         lcp[r] = h;
19         if (h) h--;
20     }
21     return lcp;
22 }

```

4 Graph

Checklist: directed or undirected? weights negative? multiple edges/self-loops?
clear globals between tests? recursion depth? 0-index input conversion?

4.1 Dijkstra

Use: Single-source shortest paths with nonnegative weights.

Complexity: $O((n + m) \log n)$.

Pitfalls: No negative edges. INFL must exceed max path; d+w can overflow if too close.

Assumptions/deps: Graph as `vector<vector<pair<int,ll>>>`.

```
1 // Dijkstra shortest paths for nonnegative edge weights.
2 // Complexity:  $O((n + m) \log n)$ .
3 // Pitfalls: no negative edges; INF must exceed any possible distance.
4 // Usage: vector<vector<pair<int,ll>>> g(n); auto d = dijkstra(g, s);
5 using ll = long long;
6 const ll INFL = (1LL << 62);
7
8 vector<ll> dijkstra(const vector<vector<pair<int, ll>>>& g, int s) {
9     int n = (int)g.size();
10    vector<ll> dist(n, INFL);
11    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
12    dist[s] = 0;
13    pq.push({0, s});
14    while (!pq.empty()) {
15        auto [d, v] = pq.top();
16        pq.pop();
17        if (d != dist[v]) continue;
18        for (auto [to, w] : g[v]) {
19            if (dist[to] > d + w) {
20                dist[to] = d + w;
21                pq.push({dist[to], to});
22            }
23        }
24    }
25    return dist;
26 }
```

4.2 Strongly Connected Components

Use: Kosaraju SCC decomposition.

Complexity: $O(n + m)$.

Pitfalls: Recursive DFS may overflow. Component ids are source-to-sink topological order of SCC DAG.

Assumptions/deps: Directed graph; call `init` per test case.

```
1 // Strongly connected components, Kosaraju.
2 // Complexity:  $O(n + m)$ .
3 // Pitfalls: comp ids are topological order of SCC DAG sources to sinks.
4 // Usage: SCC s(n); s.addEdge(u,v); int c = s.build(); s.comp[v] in [0,c).
5 struct SCC {
6     int n;
7     vector<vector<int>> g, rg;
8     vector<int> comp;
9     SCC(int n = 0) { init(n); }
10    void init(int n_) {
11        n = n_;
12        g.assign(n, {});
13        rg.assign(n, {});
14        comp.assign(n, -1);
15    }
16    void addEdge(int u, int v) {
17        g[u].push_back(v);
18        rg[v].push_back(u);
19    }
20    int build() {
21        vector<int> order, vis(n);
22        function<void(int)> dfs1 = [&](int v) {
23            vis[v] = 1;
24            for (int to : g[v]) if (!vis[to]) dfs1(to);
25            order.push_back(v);
26        };
27        function<void(int, int)> dfs2 = [&](int v, int c) {
28            comp[v] = c;
29            for (int to : rg[v]) if (comp[to] == -1) dfs2(to, c);
30        };
31        for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
32        reverse(order.begin(), order.end());
33        int cnt = 0;
34        for (int v : order) if (comp[v] == -1) dfs2(v, cnt++);
35        return cnt;
36    }
37 };
```

4.3 LCA Binary Lifting

Use: LCA and distance in rooted tree.

Complexity: Build $O(n \log n)$, query $O(\log n)$.

Pitfalls: Call `build(root)`; recursion depth can overflow on large chain.

Assumptions/deps: Graph must be a tree; 0-indexed.

```
1 // Binary lifting LCA for rooted trees.
2 // Complexity: build  $O(n \log n)$ , query  $O(\log n)$ .
3 // Pitfalls: call build(root); graph must be a tree; recursion depth on large chains.
```

```

4 // Usage: LCA lca(n); add undirected edges to lca.g; lca.build(0); lca.lca(u,v).
5 struct LCA {
6     int n, LOG;
7     vector<vector<int>> g, up;
8     vector<int> depth;
9     LCA(int n = 0) { init(n); }
10    void init(int n_) {
11        n = n_;
12        LOG = 1;
13        while ((1 << LOG) <= max(1, n)) LOG++;
14        g.assign(n, {});
15        up.assign(LOG, vector<int>(n));
16        depth.assign(n, 0);
17    }
18    void addEdge(int u, int v) {
19        g[u].push_back(v);
20        g[v].push_back(u);
21    }
22    void dfs(int v, int p) {
23        up[0][v] = p;
24        for (int j = 1; j < LOG; j++) up[j][v] = up[j - 1][up[j - 1][v]];
25        for (int to : g[v]) if (to != p) {
26            depth[to] = depth[v] + 1;
27            dfs(to, v);
28        }
29    }
30    void build(int root = 0) {
31        depth[root] = 0;
32        dfs(root, root);
33    }
34    int lift(int v, int k) {
35        for (int j = 0; j < LOG; j++) if (k >> j & 1) v = up[j][v];
36        return v;
37    }
38    int lca(int a, int b) {
39        if (depth[a] < depth[b]) swap(a, b);
40        a = lift(a, depth[a] - depth[b]);
41        if (a == b) return a;
42        for (int j = LOG - 1; j >= 0; j--) if (up[j][a] != up[j][b]) {
43            a = up[j][a];
44            b = up[j][b];
45        }
46        return up[0][a];
47    }
48    int dist(int a, int b) {
49        int c = lca(a, b);
50        return depth[a] + depth[b] - 2 * depth[c];
51    }
52 };

```

4.4 Bridges

Use: Find bridges in undirected graph, including multiedges.

Complexity: $O(n + m)$.

Pitfalls: Parent edge skip handles one parallel edge; remember graph is undirected.

Assumptions/deps: Define IS_BRIDGE; globals n, adj.

```

1 void IS_BRIDGE(int v, int to); // some function to process the found bridge. Printing
  for instance
2 int n; // number of nodes
3 vector<vector<int>> adj; // adjacency list of graph
4
5 vector<bool> visited;
6 vector<int> tin, low;
7 int timer;
8
9 void dfs(int v, int p = -1) {
10     visited[v] = true;
11     tin[v] = low[v] = timer++;
12     bool parent_skipped = false;
13     for (int to : adj[v]) {
14         if (to == p && !parent_skipped) {
15             parent_skipped = true;
16             continue;
17         }
18         if (visited[to]) {
19             low[v] = min(low[v], tin[to]);
20         } else {
21             dfs(to, v);
22             low[v] = min(low[v], low[to]);
23             if (low[to] > tin[v])
24                 IS_BRIDGE(v, to);
25         }
26     }
27 }
28
29 void find_bridges() {
30     timer = 0;
31     visited.assign(n, false);
32     tin.assign(n, -1);
33     low.assign(n, -1);
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i])
36             dfs(i);
37     }
38 }

```

4.5 Articulation Points

Use: Find cut vertices in undirected graph.

Complexity: $O(n + m)$.

Pitfalls: Root needs more than one DFS child. Current version can report same vertex multiple times.

Assumptions/deps: Define IS_CUTPOINT; globals n, adj.

```
1 void IS_CUTPOINT(int v); // here v is articulation point, i can print for instance.
2 int n; // number of nodes
3 vector<vector<int>> adj; // adjacency list of graph
4
5 vector<bool> visited;
6 vector<int> tin, low;
7 int timer;
8
9 void dfs(int v, int p = -1) {
10     visited[v] = true;
11     tin[v] = low[v] = timer++;
12     int children=0;
13     for (int to : adj[v]) {
14         if (to == p) continue;
15         if (visited[to]) {
16             low[v] = min(low[v], tin[to]);
17         } else {
18             dfs(to, v);
19             low[v] = min(low[v], low[to]);
20             if (low[to] >= tin[v] && p != -1)
21                 IS_CUTPOINT(v);
22             ++children;
23         }
24     }
25     if (p == -1 && children > 1)
26         IS_CUTPOINT(v);
27 }
28
29 void find_cutpoints() {
30     timer = 0;
31     visited.assign(n, false);
32     tin.assign(n, -1);
33     low.assign(n, -1);
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i])
36             dfs(i);
37     }
38 }
```

4.6 Dinic Max Flow

Use: Directed max flow and min cut.

Complexity: General fast in practice; unit capacities have better listed bounds.

Pitfalls: Edges are directed; add both directions for undirected capacity. Capacities are long long.

Assumptions/deps: Requires sz, rep, bits/stdc++.h/STL.

```
1 // * $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
2 // After calc(s,t), leftOfMinCut(v) gives the source side; minCutEdges() lists
   crossing edges.
3 // Requires bits/stdc++.h and using namespace std.
4 using ll = long long;
5 using vi = vector<int>;
6 #ifndef sz
7 #define sz(x) (int)(x).size()
8 #endif
9 #ifndef rep
10 #define rep(i,a,b) for (int i = (a); i < (b); ++i)
11 #endif
12
13 struct Dinic {
14     struct Edge {
15         int to, rev;
16         ll c, oc;
17         ll flow() { return max(oc - c, 0LL); } // if you need flows
18     };
19     vi lvl, ptr, q;
20     vector<vector<Edge>> adj;
21     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
22     void addEdge(int a, int b, ll c, ll rcap = 0) {
23         adj[a].push_back({b, sz(adj[b]), c, c});
24         adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
25     }
26     ll dfs(int v, int t, ll f) {
27         if (v == t || !f) return f;
28         for (int& i = ptr[v]; i < sz(adj[v]); i++) {
29             Edge& e = adj[v][i];
30             if (lvl[e.to] == lvl[v] + 1)
31                 if (ll p = dfs(e.to, t, min(f, e.c))) {
32                     e.c -= p, adj[e.to][e.rev].c += p;
33                     return p;
34                 }
35         }
36         return 0;
37     }
38     ll calc(int s, int t) {
39         ll flow = 0; q[0] = s;
40         rep(L,0,31) do { // 'int L=30' maybe faster for random data
41             lvl = ptr = vi(sz(q));
42             int qi = 0, qe = lvl[s] = 1;
```

```

43 while (qi < qe && !lvl[t]) {
44     int v = q[qi++];
45     for (Edge e : adj[v])
46         if (!lvl[e.to] && e.c >> (30 - L))
47             q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
48 }
49 while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
50 } while (lvl[t]);
51 return flow;
52 }
53 bool leftOfMinCut(int a) { return lvl[a] != 0; }
54 vector<pair<int, int>> minCutEdges() {
55     vector<pair<int, int>> cut;
56     rep(v, 0, sz(adj)) if (leftOfMinCut(v)) {
57         for (Edge& e : adj[v])
58             if (e.oc > 0 && e.c == 0 && !leftOfMinCut(e.to))
59                 cut.push_back({v, e.to});
60     }
61     return cut;
62 }
63 };

```

4.7 Min Cost Flow

Use: Successive shortest path with potentials.

Complexity: $O(FE \log V)$ plus Bellman-Ford init.

Pitfalls: Negative costs ok, but no negative cost cycles. May return less than requested flow.

Assumptions/deps: Cap/cost are long long; directed edges.

```

1 // Min-cost flow with potentials. Negative costs ok; no negative cost cycles.
2 // Source: Stanford/KACTL successive shortest path style.
3 // Complexity:  $O(F E \log V)$ , plus  $O(VE)$  Bellman-Ford init.
4 // Pitfalls: costs/caps are long long; minCost(s,t,need) may send < need flow.
5 // Usage: MCMF mf(n); mf.addEdge(u,v,cap,cost); auto [flow,cost]=mf.minCost(s,t,need)
6 ;
7 struct MCMF {
8     using ll = long long;
9     const ll INF = (1LL << 62);
10     struct Edge {
11         int to, rev;
12         ll cap, cost;
13     };
14     int n;
15     vector<vector<Edge>> g;
16     vector<ll> pot, dist;
17     vector<int> pv, pe;

```

```

18 MCMF(int n = 0) { init(n); }
19 void init(int n_) {
20     n = n_;
21     g.assign(n, {});
22     pot.assign(n, 0);
23     dist.resize(n);
24     pv.resize(n);
25     pe.resize(n);
26 }
27 void addEdge(int v, int to, ll cap, ll cost) {
28     if (cap <= 0) return;
29     Edge a{to, (int)g[to].size(), cap, cost};
30     Edge b{v, (int)g[v].size(), 0, -cost};
31     g[v].push_back(a);
32     g[to].push_back(b);
33 }
34 void setPotentials(int s) {
35     fill(pot.begin(), pot.end(), INF);
36     pot[s] = 0;
37     for (int it = 0; it < n - 1; it++) {
38         bool changed = false;
39         for (int v = 0; v < n; v++) if (pot[v] != INF) {
40             for (Edge& e : g[v]) if (e.cap && pot[e.to] > pot[v] + e.cost) {
41                 pot[e.to] = pot[v] + e.cost;
42                 changed = true;
43             }
44         }
45         if (!changed) break;
46     }
47     for (ll& x : pot) if (x == INF) x = 0;
48 }
49 pair<ll, ll> minCost(int s, int t, ll need = (1LL << 62)) {
50     setPotentials(s);
51     ll flow = 0, cost = 0;
52     while (flow < need) {
53         fill(dist.begin(), dist.end(), INF);
54         priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
55         dist[s] = 0;
56         pq.push({0, s});
57         while (!pq.empty()) {
58             auto [d, v] = pq.top();
59             pq.pop();
60             if (d != dist[v]) continue;
61             for (int i = 0; i < (int)g[v].size(); i++) {
62                 Edge& e = g[v][i];
63                 if (!e.cap) continue;
64                 ll nd = d + e.cost + pot[v] - pot[e.to];
65                 if (nd < dist[e.to]) {
66                     dist[e.to] = nd;
67                     pv[e.to] = v;
68                     pe[e.to] = i;

```

```

69         pq.push({nd, e.to});
70     }
71 }
72 }
73 if (dist[t] == INF) break;
74 for (int v = 0; v < n; v++) if (dist[v] != INF) pot[v] += dist[v];
75 ll add = need - flow;
76 for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
77 for (int v = t; v != s; v = pv[v]) {
78     Edge& e = g[pv[v]][pe[v]];
79     e.cap -= add;
80     g[v][e.rev].cap += add;
81     cost += add * e.cost;
82 }
83 flow += add;
84 }
85 return {flow, cost};
86 }
87 };

```

4.8 Kuhn Bipartite Matching

Use: Maximum bipartite matching for smaller/simple cases.

Complexity: Usually $O(VE)$ worst case.

Pitfalls: Left side size n , right side m . Rebuild visited each phase.

Assumptions/deps: Use Hopcroft-Karp if limits are large.

```

1 struct bipartite {
2     int n, m;
3     vector<vector<int>> g;
4     vector<int> match;
5
6     bipartite(int n, int m): n(n), m(m), g(n), match(m, -1) {}
7
8     void add(int a, int b) {
9         g[a].push_back(b);
10    }
11
12    vector<bool> visited;
13    bool kuhn(int v) {
14        if(visited[v]) {
15            return false;
16        }
17        visited[v] = true;
18        for(auto u: g[v]) {
19            if(match[u] == -1 || kuhn(match[u])) {
20                match[u] = v;
21                return true;

```

```

22    }
23    }
24    return false;
25 }
26
27 auto matching() {
28     vector<bool> paired(n);
29     for(bool repeat = true; repeat;) {
30         repeat = false;
31         visited.assign(n, 0);
32         for(int i = 0; i < n; i++) {
33             if(!paired[i]) {
34                 paired[i] = kuhn(i);
35                 repeat |= paired[i];
36             }
37         }
38     }
39     vector<pair<int, int>> ans;
40     for(int u = 0; u < m; u++) {
41         if(match[u] != -1) {
42             ans.emplace_back(match[u], u);
43         }
44     }
45     return ans;
46 }
47 };

```

4.9 Heavy Light Decomposition

Use: Path/subtree add and max query on tree.

Complexity: $O(\log^2 n)$ path operations.

Pitfalls: Constructor roots at 0 and mutates adjacency. VALS_EDGES=true excludes LCA.

Assumptions/deps: Includes dynamic segment tree Node; requires macros sz/all.

```

1 using vi = vector<int>;
2 #ifndef sz
3 #define sz(x) (int)(x).size()
4 #endif
5 #ifndef all
6 #define all(x) begin(x), end(x)
7 #endif
8
9 // Dependency used by HLD below: range add, range max, half-open [l, r).
10 struct Node {
11     int lo, hi, lazy = 0, val = 0;
12     Node *l = 0, *r = 0;

```

```

13 Node(int lo, int hi) : lo(lo), hi(hi) {}
14 void push() {
15     if (hi - lo <= 1) return;
16     int mid = (lo + hi) / 2;
17     if (!l) l = new Node(lo, mid), r = new Node(mid, hi);
18     if (lazy) {
19         l->add(lo, hi, lazy);
20         r->add(lo, hi, lazy);
21         lazy = 0;
22     }
23 }
24 void add(int L, int R, int x) {
25     if (R <= lo || hi <= L) return;
26     if (L <= lo && hi <= R) {
27         val += x;
28         lazy += x;
29         return;
30     }
31     push();
32     l->add(L, R, x);
33     r->add(L, R, x);
34     val = max(l->val, r->val);
35 }
36 int query(int L, int R) {
37     if (R <= lo || hi <= L) return INT_MIN;
38     if (L <= lo && hi <= R) return val;
39     push();
40     return max(l->query(L, R), r->query(L, R));
41 }
42 };
43
44 // 0-indexed. VALS_EDGES=true stores edge values at the deeper endpoint.
45 // Path/subtree operations are O(log^2 N) with the Node above.
46 template <bool VALS_EDGES> struct HLD {
47     int N, tim = 0;
48     vector<vi> adj;
49     vi par, siz, rt, pos;
50     Node *tree;
51     HLD(vector<vi> adj_)
52         : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
53           rt(N), pos(N), tree(new Node(0, N)) { dfsSz(0); dfsHld(0); }
54     void dfsSz(int v) {
55         if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
56         for (int& u : adj[v]) {
57             par[u] = v;
58             dfsSz(u);
59             siz[v] += siz[u];
60             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
61         }
62     }
63     void dfsHld(int v) {
64         pos[v] = tim++;

```

```

65         for (int u : adj[v]) {
66             rt[u] = (u == adj[v][0] ? rt[v] : u);
67             dfsHld(u);
68         }
69     }
70     template <class B> void process(int u, int v, B op) {
71         for (; rt[u] != rt[v]; v = par[rt[v]]) {
72             if (pos[rt[u]] > pos[rt[v]]) swap(u, v);
73             op(pos[rt[v]], pos[v] + 1);
74         }
75         if (pos[u] > pos[v]) swap(u, v);
76         op(pos[u] + VALS_EDGES, pos[v] + 1);
77     }
78     void modifyPath(int u, int v, int val) {
79         process(u, v, [&](int l, int r) { tree->add(l, r, val); });
80     }
81     int queryPath(int u, int v) {
82         int res = INT_MIN;
83         process(u, v, [&](int l, int r) { res = max(res, tree->query(l, r)); });
84         return res;
85     }
86     void modifySubtree(int v, int val) {
87         tree->add(pos[v] + VALS_EDGES, pos[v] + siz[v], val);
88     }
89     int querySubtree(int v) {
90         return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
91     }
92 };
93 // op(pos[u] + VALS_EDGES, pos[v] + 1);
94 // - HLD<false>: values are on vertices.
95 // - Path u..v includes both endpoints.
96 // - Query/update range starts at pos[u].
97 // - HLD<true>: values are on edges.
98 // - Store each edge's value at the deeper child vertex.
99 // - Path u..v should exclude the LCA vertex, because no edge is stored 'on' the
100 //   LCA.
101 // - So range starts at pos[u] + 1.
102 // Example: edge values on path 2 -> 1 -> 3.
103 // The LCA is 1. The edges are represented at vertices 2 and 3, not at 1, so
104 //   VALS_EDGES=true skips pos[1].

```

4.10 DSU on Tree Sack

Use: Answer subtree color-frequency queries.

Complexity: $O(n \log n)$ total for add/remove loops.

Pitfalls: Reset all globals per test; recursion depth on chains; color ids must fit cnt.

Assumptions/deps: Tree in global g; fill answer at marked line.


```

1 // DSU on tree / sack for subtree color counts.
2 // Fill g[0..n), col[v]. Call dfs_sz(root,-1), then dfs(root,-1,true).
3 // In dfs(), cnt[c] is the count of color c in subtree(v) at the answer point.
4 // Time: O(n log n) total for add/remove loops.
5 const int MAXN = 200000 + 5;
6 vector<int> g[MAXN];
7 int col[MAXN], sz[MAXN], st[MAXN], ft[MAXN], ver[MAXN], cnt[MAXN], timer_sack;
8
9 void dfs_sz(int v, int p) {
10     st[v] = timer_sack;
11     ver[timer_sack++] = v;
12     sz[v] = 1;
13     for (int u : g[v]) if (u != p) {
14         dfs_sz(u, v);
15         sz[v] += sz[u];
16     }
17     ft[v] = timer_sack;
18 }
19
20 void add_subtree(int v, int delta) {
21     for (int i = st[v]; i < ft[v]; i++)
22         cnt[col[ver[i]]] += delta;
23 }
24
25 void dfs(int v, int p, bool keep) {
26     int bigChild = -1;
27     for (int u : g[v])
28         if (u != p && (bigChild == -1 || sz[u] > sz[bigChild]))
29             bigChild = u;
30
31     for (int u : g[v])
32         if (u != p && u != bigChild)
33             dfs(u, v, false);
34
35     if (bigChild != -1) dfs(bigChild, v, true);
36
37     for (int u : g[v])
38         if (u != p && u != bigChild)
39             add_subtree(u, +1);
40     cnt[col[v]]++;
41
42     // answer queries for v here, using cnt[color]
43
44     if (!keep) add_subtree(v, -1);
45 }

```

4.11 Virtual Tree

Use: Build compressed tree from key vertices and LCAs.

Complexity: $O(k \log k)$ after LCA preprocessing.

Pitfalls: Snippet is a full problem solution; clears only virtual nodes in result. LG must fit n.

Assumptions/deps: Depends on its own DFS/LCA globals; 0-indexed tree.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 constexpr int MAX_N = 2e5 + 1;
7 constexpr int LG = 18;
8 constexpr int MOD = 998244353;
9
10 int n, a[MAX_N];
11 ll dp[MAX_N], ans;
12 int tin[MAX_N], tout[MAX_N], d[MAX_N], lift[MAX_N][LG], timer;
13 vector<int> adj[MAX_N], vadj[MAX_N], at_a[MAX_N];
14
15 void dfs(int v, int p) {
16     at_a[a[v]].push_back(v);
17     tin[v] = timer++;
18
19     lift[v][0] = p;
20     for (int i = 1; i < LG; i++) { lift[v][i] = lift[lift[v][i - 1]][i - 1]; }
21
22     for (int u : adj[v]) {
23         if (u == p) { continue; }
24         dfs(u, v);
25     }
26
27     tout[v] = timer++;
28 }
29
30 int is_ancestor(int u, int v) { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
31
32 int lca(int u, int v) {
33     if (is_ancestor(u, v)) { return u; }
34     if (is_ancestor(v, u)) { return v; }
35
36     for (int i = LG - 1; i >= 0; i--) {
37         if (!is_ancestor(lift[u][i], v)) { u = lift[u][i]; }
38     }
39     return lift[u][0];
40 }
41
42 bool sort_tin(const int &a, const int &b) { return tin[a] < tin[b]; }
43

```

```

44 vector<int> vtree(const vector<int> &key) {
45     // Returns key vertices plus needed LCAs, sorted by tin.
46     if (key.empty()) return {};
47
48     vector<int> res = key;
49     sort(res.begin(), res.end(), sort_tin);
50
51     int k = res.size();
52     for (int i = 1; i < k; i++) {
53         res.push_back(lca(res[i - 1], res[i]));
54     }
55
56     sort(res.begin(), res.end(), sort_tin);
57     res.erase(unique(res.begin(), res.end(), res.end()));
58
59     for (int v : res) { vadj[v].clear(); }
60
61     for (int i = 1; i < (int)res.size(); i++) {
62         vadj[lca(res[i - 1], res[i])].push_back(res[i]);
63     }
64
65     return res;
66 }
67
68 int main() {
69     cin >> n;
70
71     for (int i = 0; i < n; i++) { cin >> a[i]; }
72
73     for (int i = 1; i < n; i++) {
74         int u, v;
75         cin >> u >> v;
76         u--;
77         v--;
78         adj[u].push_back(v);
79         adj[v].push_back(u);
80     }
81
82     dfs(0, 0);
83
84     for (int col = 1; col <= n; col++) {
85         vector<int> vt = vtree(at_a[col]);
86         reverse(vt.begin(), vt.end());
87
88         for (int v : vt) {
89             dp[v] = 1;
90             for (int u : vadj[v]) {
91                 dp[v] *= (dp[u] + 1);
92                 dp[v] %= MOD;
93             }
94             if (a[v] != col) { dp[v]--; }
95

```

```

96         ans += dp[v];
97         ans %= MOD;
98         if (a[v] != col) {
99             for (int u : vadj[v]) {
100                 ans += MOD - dp[u];
101                 ans %= MOD;
102             }
103         }
104     }
105 }
106
107 cout << ans << '\n';
108 return 0;
109 }

```

4.12 Kruskal Reconstruction Tree

Use: Build merge tree for MST edge-threshold queries.

Complexity: $O(m \log m \alpha(n))$.

Pitfalls: For disconnected graph, returned root is only last component. MAXK must be large enough.

Assumptions/deps: Uses global edges; clear between tests.

```

1 // Kruskal reconstruction tree for an undirected weighted graph.
2 // Original vertices are [0, n0). New internal node weight is edge weight.
3 // For connected graph, root is total_nodes - 1 after build().
4 // Time:  $O(m \log m \alpha(n))$ .
5 const int MAXK = 400000 + 5; // at least 2 * original_n + m_components
6 struct KEdge { int w, u, v; };
7 vector<KEdge> edges;
8 vector<int> ktree[MAXK];
9 int dsu[MAXK], node_weight[MAXK], total_nodes;
10
11 int getRoot(int u) {
12     if (u == dsu[u]) return u;
13     return dsu[u] = getRoot(dsu[u]);
14 }
15
16 int build_kruskal_tree(int n0) {
17     total_nodes = n0;
18     for (int i = 0; i < MAXK; i++) {
19         dsu[i] = i;
20         ktree[i].clear();
21     }
22     sort(edges.begin(), edges.end(), [](const KEdge& a, const KEdge& b) {
23         return a.w < b.w;
24     });
25     for (auto e : edges) {

```

```

26     int u = getRoot(e.u), v = getRoot(e.v);
27     if (u == v) continue;
28     int x = total_nodes++;
29     node_weight[x] = e.w;
30     ktree[x].push_back(u);
31     ktree[x].push_back(v);
32     dsu[u] = dsu[v] = dsu[x] = x;
33 }
34 return total_nodes - 1;
35 }

```

4.13 Global Min Cut

Use: Stoer-Wagner undirected global min cut.

Complexity: $O(V^3)$ with matrix version.

Pitfalls: Undirected symmetric matrix only; weights nonnegative; integer sums may overflow.

Assumptions/deps: Requires vi, sz, rep, all.

```

1 pair<int, vi> globalMinCut(vector<vi> mat) {
2     pair<int, vi> best = {INT_MAX, {}};
3     int n = sz(mat);
4     vector<vi> co(n);
5     rep(i,0,n) co[i] = {i};
6     rep(ph,1,n) {
7         vi w = mat[0];
8         size_t s = 0, t = 0;
9         rep(it,0,n-ph) { //  $O(V^2)$  →  $O(E \log V)$  with prio. queue
10             w[t] = INT_MIN;
11             s = t, t = max_element(all(w)) - w.begin();
12             rep(i,0,n) w[i] += mat[t][i];
13         }
14         best = min(best, {w[t] - mat[t][t], co[t]});
15         co[s].insert(co[s].end(), all(co[t]));
16         rep(i,0,n) mat[s][i] += mat[t][i];
17         rep(i,0,n) mat[i][s] = mat[s][i];
18         mat[0][t] = INT_MIN;
19     }
20     return best;
21 }

```

4.14 Maximum Clique

Use: Branch-and-bound maximum clique / complement for independent set.

Complexity: Exponential; fast for about $n \leq 155$ in tested cases.

Pitfalls: Input must be symmetric bitset matrix, no self-edges. bitset<200> limits n.

Assumptions/deps: Requires vi, sz, all, rep.

```

1 /**
2  * Author: chilli, SJTU, Janez Konc
3  * Date: 2019-05-10
4  * License: GPL3+
5  * Source: https://en.wikipedia.org/wiki/MaxCliqueDyn\_maximum\_clique\_algorithm, https://gitlab.com/janezkonc/mcqd/blob/master/mcqd.h
6  * Description: Quickly finds a maximum clique of a graph (given as symmetric bitset
7  * matrix; self-edges not allowed). Can be used to find a maximum independent
8  * set by finding a clique of the complement graph.
9  * Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs
10 * faster for sparse graphs.
11 * Status: stress-tested
12 */
13 typedef vector<bitset<200>> vb;
14 struct Maxclique {
15     double limit=0.025, pk=0;
16     struct Vertex { int i, d=0; };
17     typedef vector<Vertex> vv;
18     vb e;
19     vv V;
20     vector<vi> C;
21     vi qmax, q, S, old;
22     void init(vv& r) {
23         for (auto& v : r) v.d = 0;
24         for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
25         sort(all(r), [](auto a, auto b) { return a.d > b.d; });
26         int mxD = r[0].d;
27         rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
28     }
29     void expand(vv& R, int lev = 1) {
30         S[lev] += S[lev - 1] - old[lev];
31         old[lev] = S[lev - 1];
32         while (sz(R)) {
33             if (sz(q) + R.back().d <= sz(qmax)) return;
34             q.push_back(R.back().i);
35             vv T;
36             for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
37             if (sz(T)) {
38                 if (S[lev]++ / ++pk < limit) init(T);
39                 int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
40                 C[1].clear(), C[2].clear();
41                 for (auto v : T) {
42                     int k = 1;
43                     auto f = [&](int i) { return e[v.i][i]; };
44                     while (any_of(all(C[k]), f)) k++;
45                     if (k > mxk) mxk = k, C[mxk + 1].clear();
46                     if (k < mnk) T[j++] .i = v.i;
47                     C[k].push_back(v.i);

```

```

48     }
49     if (j > 0) T[j - 1].d = 0;
50     rep(k,mnk,mxk + 1) for (int i : C[k])
51         T[j].i = i, T[j++].d = k;
52     expand(T, lev + 1);
53 } else if (sz(q) > sz(qmax)) qmax = q;
54 q.pop_back(), R.pop_back();
55 }
56 }
57 vi maxClique() { init(V), expand(V); return qmax; }
58 Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
59     rep(i,0,sz(e)) V.push_back({i});
60 }
61 };

```

4.15 Critical Cities Dominator

Use: Dominator-style critical vertices from source to sink in directed graph.

Complexity: About $O((n + m) \log n)$.

Pitfalls: Standalone problem solution, not a reusable library. Assumes sink reachable from source.

Assumptions/deps: Includes its own main; 1-indexed input converted to 0-index.

```

1 #include <algorithm>
2 #include <cassert>
3 #include <cmath>
4 #include <cstdint>
5 #include <functional>
6 #include <iostream>
7 #include <vector>
8
9 using std::cout;
10 using std::endl;
11 using std::vector;
12
13 // BeginCodeSnip{Tree Class for rdom}
14 template <typename T> class Tree {
15     private:
16     const int root = 0;
17     const vector<int> &parents;
18     const vector<T> &vals;
19     const int log2dist;
20     vector<vector<int>> pow2ends;
21     vector<vector<T>> pow2mins;
22     vector<int> depth;
23
24     public:
25     Tree(const vector<int> &parents, const vector<T> &vals)

```

```

26     : parents(parents), vals(vals),
27     log2dist(std::ceil(std::log2(parents.size() + 1))),
28     pow2ends(parents.size(), vector<int>(log2dist + 1)),
29     pow2mins(parents.size(), vector<T>(log2dist + 1)) {
30     assert(parents[root] == -1);
31
32     vector<vector<int>> children(parents.size());
33     for (int n = 0; n < parents.size(); n++) {
34         if (parents[n] != -1) { children[parents[n]].push_back(n); }
35     }
36     depth = vector<int>(parents.size());
37     vector<int> frontier{root};
38     while (!frontier.empty()) {
39         int curr = frontier.back();
40         frontier.pop_back();
41         for (int n : children[curr]) {
42             depth[n] = depth[curr] + 1;
43             frontier.push_back(n);
44         }
45     }
46
47     for (int n = 0; n < parents.size(); n++) {
48         pow2mins[n][0] = vals[n];
49         pow2ends[n][0] = parents[n];
50     }
51     for (int p = 1; p <= log2dist; p++) {
52         for (int n = 0; n < parents.size(); n++) {
53             int halfway = pow2ends[n][p - 1];
54             if (halfway == -1) {
55                 pow2ends[n][p] = -1;
56                 pow2mins[n][p] = pow2mins[n][p - 1];
57             } else {
58                 pow2ends[n][p] = pow2ends[halfway][p - 1];
59                 pow2mins[n][p] =
60                     std::min(pow2mins[n][p - 1], pow2mins[halfway][p - 1]);
61             }
62         }
63     }
64 }
65
66 /**
67  * @return the min value on the path from the ancestor to its descendant,
68  * not including the ancestor.
69  */
70 T min_val(int ancestor, int desc) {
71     int dist = depth[desc] - depth[ancestor];
72     T ret = vals[desc];
73     int at = desc;
74     for (int pow = 0; pow <= log2dist; pow++) {
75         if ((dist & (1 << pow)) != 0) {
76             ret = std::min(ret, pow2mins[at][pow]);
77             at = pow2ends[at][pow];

```

```

78     }
79 }
80 assert(at == ancestor);
81 return ret;
82 }
83 };
84 // EndCodeSnip
85
86 int main() {
87     int city_num;
88     int flight_num;
89     std::cin >> city_num >> flight_num;
90     vector<vector<int>> adj(city_num);
91     vector<vector<int>> rev_adj(city_num);
92     for (int f = 0; f < flight_num; f++) {
93         int a, b;
94         std::cin >> a >> b;
95         adj[--a].push_back(--b);
96         rev_adj[b].push_back(a);
97     }
98
99     // Variables used for the Euler Tour
100     vector<int> visit_order;
101     // visit_time is initialized with INF so nodes that aren't reachable
102     // during the DFS won't mess up our sdом calculation
103     vector<int> visit_time(city_num, INT32_MAX);
104     vector<int> visit_parent(city_num, -1);
105     vector<bool> visited(city_num, false);
106     std::function<void(int, int)> dfs;
107     dfs = [&](int at, int prev) {
108         if (visited[at]) { return; }
109         visited[at] = true;
110
111         visit_time[at] = visit_order.size();
112         visit_order.push_back(at);
113         visit_parent[at] = prev;
114         for (int next : adj[at]) { dfs(next, at); }
115     };
116     dfs(0, -1);
117
118     // We use a function-based interface instead of a class-based one due to
119     // heavy reliance on previously calculated values
120     vector<int> dsu_parent(city_num, -1);
121     vector<int> dsu_min(city_num, INT32_MAX);
122     std::function<int(int)> find;
123     find = [&](int x) {
124         if (dsu_parent[x] == -1) { return x; }
125         if (dsu_parent[dsu_parent[x]] != -1) {
126             int parent_res = find(dsu_parent[x]);
127             if (visit_time[parent_res] < visit_time[dsu_min[x]]) {
128                 dsu_min[x] = parent_res;
129             }

```

```

130         dsu_parent[x] = dsu_parent[dsu_parent[x]];
131     }
132     assert(dsu_min[x] != INT32_MAX);
133     return dsu_min[x];
134 };
135
136 vector<int> sdом(city_num, -1);
137 for (int i = visit_order.size() - 1; i > 0; i--) {
138     int c = visit_order[i];
139     // Iterate over all nodes with a directed edge to c
140     for (int from : rev_adj[c]) {
141         int find_res = find(from);
142         // Take the node with the earliest visit time in the traversal
143         if (sdом[c] == -1 || visit_time[find_res] < visit_time[sdom[c]]) {
144             sdом[c] = find_res;
145         }
146     }
147
148     // Link c to its parent in the DFS tree
149     dsu_parent[c] = visit_parent[c];
150     dsu_min[c] = sdом[c];
151 }
152
153 // Initialize the values in the tree
154 vector<std::pair<int, int>> sdом_times(city_num, {INT32_MAX, -1});
155 for (int c : visit_order) {
156     if (c != 0) {
157         // The tree takes the minimum of these pairs, but what we
158         // really want is the node itself- therefore, we store both
159         sdом_times[c] = {visit_time[sdom[c]], c};
160     }
161 }
162 Tree tree(visit_parent, sdом_times);
163
164 vector<int> rdom(city_num, -1);
165 for (int c : visit_order) {
166     if (c != 0) {
167         // Use the definition of the rdom
168         rdom[c] = tree.min_val(sdom[c], c).second;
169     }
170 }
171
172 // Use the properties previously given to compute the idom
173 vector<int> idом(city_num, -1);
174 for (int c : visit_order) {
175     if (c != 0) { idом[c] = rdom[c] == c ? sdом[c] : idом[rdom[c]]; }
176 }
177
178 // Trace the idом tree from the finish node to the start node,
179 // finding all dominators along the way
180 // NOTE: assumes city_num-1 is reachable from 0; otherwise idом[-1] UB.
181 vector<int> critical{0};

```

```

182 int at = city_num - 1;
183 while (at != 0) {
184     critical.push_back(at);
185     at = idom[at];
186 }
187
188 std::sort(critical.begin(), critical.end());
189
190 cout << critical.size() << '\n';
191 for (int i = 0; i < critical.size() - 1; i++) { cout << critical[i] + 1 << ' '; }
192 cout << critical.back() + 1 << endl;
193 }

```

5 Math

Checklist: prime modulus? overflow in multiplication/lcm? zero inverse? random/probabilistic? clear bases/sieves? CRT residues compatible?

5.1 Number Theory Essentials

Use: Extended gcd, modular inverse, fast power.

Complexity: $O(\log n)$.

Pitfalls: inv uses Fermat: only prime modulus. Use inv_any for coprime composite mod.

Assumptions/deps: modpow uses __int128.

```

1 long long egcd(long long a, long long b, long long& x, long long& y) {
2     if (!b) {
3         x = 1; y = 0;
4         return a;
5     }
6     long long x1, y1;
7     long long g = egcd(b, a % b, x1, y1);
8     x = y1;
9     y = x1 - y1 * (a / b);
10    return g;
11 }
12
13 long long inv_any(long long a, long long mod) {
14     long long x, y;
15     long long g = egcd(a, mod, x, y);
16     if (g != 1) return -1; // no inverse
17     x %= mod;
18     if (x < 0) x += mod;

```

```

19     return x;
20 }
21
22 long long modpow(long long a, long long e, long long mod) {
23     long long r = 1;
24     while (e) {
25         if (e & 1) r = (__int128)r * a % mod;
26         a = (__int128)a * a % mod;
27         e >>= 1;
28     }
29     return r;
30 }
31
32 long long inv(long long a, long long mod) {
33     return modpow(a, mod - 2, mod);
34 }

```

5.2 Sieve

Use: Enumerate primes below limit.

Complexity: $O(n \log \log n)$.

Pitfalls: Returns primes < lim, not <= lim. MAX_PR fixed.

Assumptions/deps: Requires vi, rep.

```

1 const int MAX_PR = 5'000'000;
2 bitset<MAX_PR> isprime;
3 vi eratosthenesSieve(int lim) {
4     isprime.set(); isprime[0] = isprime[1] = 0;
5     for (int i = 4; i < lim; i += 2) isprime[i] = 0;
6     for (int i = 3; i*i < lim; i += 2) if (isprime[i])
7         for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
8     vi pr;
9     rep(i,2,lim) if (isprime[i]) pr.push_back(i);
10    return pr;
11 }

```

5.3 CRT

Use: Merge two congruences, non-coprime allowed.

Complexity: $O(\log \min(m_1, m_2))$.

Pitfalls: Works only if residues compatible. LCM can overflow long long.

Assumptions/deps: Depends on egcd from number theory essentials.

```

1 // returns {x, lcm}; x satisfies:

```

```

2 // NOTE: lcm = m1/g*m2 can overflow long long.
3 // x = a1 mod m1
4 // x = a2 mod m2
5 // returns {-1,-1} if impossible
6 pair<long long,long long> crt(long long a1, long long m1,
7                             long long a2, long long m2) {
8     long long x, y;
9     long long g = egcd(m1, m2, x, y);
10
11     if ((a2 - a1) % g != 0) return {-1, -1};
12
13     long long lcm = m1 / g * m2;
14     __int128 t = (__int128)(a2 - a1) / g * x;
15     t %= m2 / g;
16
17     long long ans = (a1 + (__int128)m1 * t) % lcm;
18     if (ans < 0) ans += lcm;
19
20     return {ans, lcm};
21 }

```

5.4 Mod Inverse Table

Use: Inverses, factorials, inverse factorials modulo prime.

Complexity: $O(n)$.

Pitfalls: MOD must be prime and $n < \text{MOD}$. ifact code is cumulative inverse factorial.

Assumptions/deps: Change MOD before use.

```

1 // Precompute modular inverses and factorial inverses modulo prime MOD.
2 // Complexity: O(n). Use when 0 <= n < MOD.
3 // Pitfalls: inv[i] uses inv[MOD % i], not inv[i-1]; MOD must be prime.
4 // Usage: build_inv(1000000); then invv[i], fact[i], ifact[i], C(n,k).
5 const long long MOD = 1000000007;
6 vector<long long> invv, fact, ifact;
7
8 void build_inv(int n) {
9     invv.assign(n + 1, 0);
10    fact.assign(n + 1, 1);
11    ifact.assign(n + 1, 1);
12    if (n >= 1) invv[1] = 1;
13    for (int i = 2; i <= n; i++)
14        invv[i] = MOD - (MOD / i) * invv[MOD % i] % MOD;
15    for (int i = 1; i <= n; i++) {
16        fact[i] = fact[i - 1] * i % MOD;
17        ifact[i] = ifact[i - 1] * invv[i] % MOD; // this one uses previous value
18    }
19 }
20

```

```

21 long long C(int n, int k) {
22     if (k < 0 || k > n) return 0;
23     return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
24 }

```

5.5 Primitive Root

Use: Find generator modulo prime.

Complexity: $O(\sqrt{p} \log p)$ roughly.

Pitfalls: Int multiplication is cast to long long, but p should fit int. Prime modulus expected.

Assumptions/deps: Uses factorization of p-1.

```

1 int powmod (int a, int b, int p) {
2     int res = 1;
3     while (b)
4         if (b & 1)
5             res = int (res * 1ll * a % p), --b;
6         else
7             a = int (a * 1ll * a % p), b >>= 1;
8     return res;
9 }
10
11 int generator (int p) {
12     vector<int> fact;
13     int phi = p-1, n = phi;
14     for (int i=2; i*i<=n; ++i)
15         if (n % i == 0) {
16             fact.push_back (i);
17             while (n % i == 0)
18                 n /= i;
19         }
20     if (n > 1)
21         fact.push_back (n);
22
23     for (int res=2; res<=p; ++res) {
24         bool ok = true;
25         for (size_t i=0; i<fact.size() && ok; ++i)
26             ok &= powmod (res, phi / fact[i], p) != 1;
27         if (ok) return res;
28     }
29     return -1;
30 }

```

5.6 Miller Rabin and Pollard Rho

Use: 64-bit primality and factorization.

Complexity: Primality $O(\log n)$; factorization expected about $O(n^{1/4})$.

Pitfalls: Probabilistic factor search but deterministic bases for 64-bit primality. Factors unsorted.

Assumptions/deps: Uses unsigned 64-bit and `__uint128_t`.

```
1 // Deterministic Miller-Rabin and Pollard Rho factorization for unsigned 64-bit.
2 // Source: KACTL MillerRabin/Factor, using __int128 mod multiplication.
3 // Complexity: primality  $O(7 \log n)$ ; factorization expected about  $O(n^{1/4})$ .
4 // Pitfalls: call factor(n) only for  $n \geq 1$ ; returns unsorted prime factors; factor
5 // Usage: isPrime(n); auto f = factor(n); sort(f.begin(), f.end());
6 using ull = unsigned long long;
7 using u128 = __uint128_t;
8
9 ull mod_mul(ull a, ull b, ull mod) {
10     return (ull)((u128)a * b % mod);
11 }
12
13 ull mod_pow(ull a, ull e, ull mod) {
14     ull r = 1;
15     while (e) {
16         if (e & 1) r = mod_mul(r, a, mod);
17         a = mod_mul(a, a, mod);
18         e >>= 1;
19     }
20     return r;
21 }
22
23 bool isPrime(ull n) {
24     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
25     ull d = n - 1, s = 0;
26     while ((d & 1) == 0) d >>= 1, s++;
27     for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022
28         ULL}) {
29         ull x = mod_pow(a % n, d, n), i = s;
30         while (x != 1 && x != n - 1 && a % n && i--> 0) x = mod_mul(x, x, n);
31         if (x != n - 1 && i != s) return false;
32     }
33     return true;
34 }
35
36 ull pollard(ull n) {
37     if (n % 2 == 0) return 2;
38     static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
39     while (true) {
40         ull c = uniform_int_distribution<ull>(1, n - 1)(rng);
41         ull x = uniform_int_distribution<ull>(0, n - 1)(rng), y = x, d = 1;
```

```
42         while (d == 1) {
43             x = f(x);
44             y = f(f(y));
45             ull diff = x > y ? x - y : y - x;
46             d = gcd(diff, n);
47         }
48         if (d != n) return d;
49     }
50 }
51
52 vector<ull> factor(ull n) {
53     if (n == 1) return {};
54     if (isPrime(n)) return {n};
55     ull d = pollard(n);
56     auto a = factor(d), b = factor(n / d);
57     a.insert(a.end(), b.begin(), b.end());
58     return a;
59 }
```

5.7 Gaussian Elimination mod p

Use: Rank of augmented matrix over prime field.

Complexity: $O(nm^2)$ style elimination.

Pitfalls: `inv()` assumes prime modulus; does not return solution vector or detect inconsistency separately.

Assumptions/deps: Depends on prime-mod inverse.

```
1 // NOTE: inv() below assumes prime mod. Composite mod needs different elimination.
2 int gauss(vector<vector<long long>> a, int mod) {
3     int n = a.size(), m = a[0].size() - 1;
4     int row = 0;
5
6     for (int col = 0; col < m && row < n; col++) {
7         int sel = row;
8         for (int i = row; i < n; i++) {
9             if (a[i][col] % mod != 0) {
10                 sel = i;
11                 break;
12             }
13         }
14         if (a[sel][col] % mod == 0) continue;
15         swap(a[sel], a[row]);
16         long long invp = inv((a[row][col] % mod + mod) % mod, mod);
17         for (int j = col; j <= m; j++) a[row][j] = a[row][j] * invp % mod;
18
19         for (int i = 0; i < n; i++) {
20             if (i != row && a[i][col]) {
21                 long long f = a[i][col];
```



```

22     for (int j = col; j <= m; j++) {
23         a[i][j] = (a[i][j] - f * a[row][j]) % mod;
24         if (a[i][j] < 0) a[i][j] += mod;
25     }
26 }
27 }
28 row++;
29 }
30 return row; // rank
31 }

```

5.8 Catalan Numbers

Use: Quick formulas for Catalan counts.

Complexity: $O(1)$ formula after binomial precompute.

Pitfalls: Division requires modular inverse; modulus must allow it.

Assumptions/deps: Use factorial/binomial table separately.

```

1 C_n = 1/(n+1) * binom(2n,n)
2 C_n = binom(2n,n) - binom(2n,n+1)

```

5.9 Mobius Sieve

Use: Linear sieve for Mobius function.

Complexity: $O(n)$.

Pitfalls: N fixed; reset arrays before each build. $\mu[1]=1$.

Assumptions/deps: Globals mu, primes, is_comp.

```

1 const int N = 1e6;
2 int mu[N + 1], primes[N + 1], pcnt;
3 bool is_comp[N + 1];
4
5 void mobius_sieve(int n) {
6     pcnt = 0;
7     for (int i = 0; i <= n; i++) {
8         mu[i] = 0;
9         is_comp[i] = false;
10    }
11    mu[1] = 1;
12    for (int i = 2; i <= n; i++) {
13        if (!is_comp[i]) {
14            primes[pcnt++] = i;
15            mu[i] = -1;
16        }
17    }
18 }

```

```

17 for (int j = 0; j < pcnt && i * primes[j] <= n; j++) {
18     int x = i * primes[j];
19     is_comp[x] = true;
20     if (i % primes[j] == 0) {
21         mu[x] = 0;
22         break;
23     } else {
24         mu[x] = -mu[i];
25     }
26 }
27 }
28 }

```

5.10 Mobius Inversion Notes

Use: Formula reminder for divisor-sum inversion.

Complexity: Reference notes.

Pitfalls: Apply only when the divisor relation really matches.

Assumptions/deps: Pair with the sieve below.

```

1 ## Möbius Function
2
3 The Möbius function  $\mu(n)$  is defined as
4 $$
5 \mu(n) =
6 \begin{cases}
7 1 & \text{if } n=1, \\
8 (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes,} \\
9 0 & \text{if } n \text{ has a squared prime factor.}
10 \end{cases}
11 $$
12
13 Key identity:
14 $$
15 \sum_{d \mid n} \mu(d) =
16 \begin{cases}
17 1 & \text{if } n=1, \\
18 0 & \text{if } n>1.
19 \end{cases}
20 $$
21 ## Inversion Formula
22
23 If
24 $$
25 F(n) = \sum_{d \mid n} f(d),
26 $$
27 then
28 $$

```

```

29 f(n) = \sum_{d \mid n} \mu(d) \cdot F \cdot \left(\frac{n}{d}\right).
30 $$

```

Equivalently, in Dirichlet convolution form:

```

33 $$
34 F = f * 1
35 \quad \Longleftrightarrow \quad
36 f = F * \mu.
37 $$

```

Common CP Applications

Counting pairs with $\gcd(i, j) = 1$:

```

42 $$
43 \sum_{d=1}^n \mu(d)
44 \left\lfloor \frac{n}{d} \right\rfloor^2
45 \quad \quad \quad
46 $$

```

Used to:

- Remove overcounting over divisors
- Transform sums over $\gcd$
- Invert divisor summations

Linear Sieve for μ ($O(n)$)

5.11 Maximum XOR Subset

Use: Linear basis maximum subset xor.

Complexity: $O(Bn)$, $B = 32$.

Pitfalls: Reset basis; signed int and bit 31 can be tricky. Use long long for bigger values.

Assumptions/deps: Standalone with main.

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define endl '\n'
5
6 int basis[35];
7 // NOTE: reset basis per test case; signed int inputs make bit 31 comparisons tricky.
8 void add (int x) {
9     for (int i = 31; i >= 0; --i) {
10         if ((x >> i) & 1) {
11             if (!basis[i]) {
12                 basis[i] = x;
13                 return;

```

```

14     }
15     x ^= basis[i];
16 }
17 }
18 }
19
20 int getMaxXor() {
21     int ans = 0;
22     for (int i = 31; i >= 0; --i) {
23         if ((ans ^ basis[i]) > ans) {
24             ans ^= basis[i];
25         }
26     }
27     return ans;
28 }
29 void solve () {
30     int n, val;
31     cin >> n;
32     for (int i = 0; i < n; ++i) {
33         cin >> val;
34         add(val);
35     }
36
37     cout << getMaxXor() << endl;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     solve();
45
46     return 0;
47 }

```

5.12 Number of Subset XOR Values

Use: Count distinct xor values from a linear basis.

Complexity: $O(Bn)$.

Pitfalls: Returns 2^{rank} , not number of subsets. Reset basis; answer may overflow int.

Assumptions/deps: Standalone with main.

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define endl '\n'
5

```

```

6 int basis[35];
7 // NOTE: counts distinct XOR values (2^rank), not subsets; reset basis per test case.
8 void add (int x) {
9     for (int i = 31; i >= 0; --i) {
10         if ((x >> i) & 1) {
11             if (!basis[i]) {
12                 basis[i] = x;
13                 return;
14             }
15             x ^= basis[i];
16         }
17     }
18 }
19
20 int getXorCount() {
21     int ans = 1;
22     for (int i = 31; i >= 0; --i) {
23         if (basis[i] > 0) {
24             ans <<= 1;
25         }
26     }
27     return ans;
28 }
29 void solve () {
30     int n, val;
31     cin >> n;
32     for (int i = 0; i < n; ++i) {
33         cin >> val;
34         add(val);
35     }
36
37     cout << getXorCount() << endl;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     solve();
45
46     return 0;
47 }

```

5.13 XOR FWHT

Use: XOR convolution.

Complexity: $O(n \log n)$.

Pitfalls: Size must be power of two. Modular inverse needed instead of division

for modular values.

Assumptions/deps: Pointwise multiply between forward and inverse transforms.

```

1 // XOR Fast Walsh-Hadamard Transform.
2 // n must be a power of two. Use inverse=true after pointwise multiply.
3 // For modular T, replace division by n with multiplication by inv(n).
4 // Time: O(n log n).
5 template<class T>
6 void fwht(vector<T>& a, bool inverse = false) {
7     int n = (int)a.size();
8     for (int len = 1; 2 * len <= n; len <= 1) {
9         for (int i = 0; i < n; i += 2 * len) {
10             for (int j = 0; j < len; j++) {
11                 T u = a[i + j], v = a[i + len + j];
12                 a[i + j] = u + v;
13                 a[i + len + j] = u - v;
14             }
15         }
16     }
17     if (inverse) {
18         for (T& x : a) x /= n;
19     }
20 }
21
22 template<class T>
23 vector<T> xor_convolution(vector<T> a, vector<T> b) {
24     fwht(a);
25     fwht(b);
26     for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
27     fwht(a, true);
28     return a;
29 }

```

5.14 Discrete kth Roots mod Prime

Use: Solve $x^k = a \pmod{p}$.

Complexity: $O(\sqrt{p} \log p)$.

Pitfalls: p must be prime. Returns distinct sorted roots; a=0 returns {0}.

Assumptions/deps: Includes primitive root and BSGS helpers.

```

1 // Solve x^k = a (mod p) for prime p.
2 // Uses primitive root + baby-step giant-step discrete log.
3 // Complexity: O(sqrt(p) log p).
4 // Pitfalls: p must be prime; returns sorted distinct roots; a=0 returns {0}.
5 // Usage: vector<long long> xs = kth_roots_mod_prime(a, k, p);
6 __extension__ typedef __int128 i128;
7
8 long long mod_pow(long long a, long long e, long long mod) {

```

```

9   long long r = 1 % mod;
10  a %= mod;
11  if (a < 0) a += mod;
12  while (e) {
13      if (e & 1) r = (i128)r * a % mod;
14      a = (i128)a * a % mod;
15      e >>= 1;
16  }
17  return r;
18 }
19
20 long long egcd(long long a, long long b, long long& x, long long& y) {
21     if (!b) return x = 1, y = 0, a;
22     long long x1, y1, g = egcd(b, a % b, x1, y1);
23     x = y1;
24     y = x1 - y1 * (a / b);
25     return g;
26 }
27
28 long long inv_mod(long long a, long long mod) {
29     long long x, y, g = egcd(a, mod, x, y);
30     if (g != 1) return -1;
31     x %= mod;
32     if (x < 0) x += mod;
33     return x;
34 }
35
36 long long primitive_root_prime(long long p) {
37     if (p == 2) return 1;
38     vector<long long> fac;
39     long long n = p - 1;
40     for (long long d = 2; d * d <= n; d++) if (n % d == 0) {
41         fac.push_back(d);
42         while (n % d == 0) n /= d;
43     }
44     if (n > 1) fac.push_back(n);
45     for (long long g = 2; ; g++) {
46         bool ok = true;
47         for (long long q : fac) if (mod_pow(g, (p - 1) / q, p) == 1) {
48             ok = false;
49             break;
50         }
51         if (ok) return g;
52     }
53 }
54
55 long long discrete_log_prime_root(long long g, long long a, long long p) {
56     // g must be a primitive root modulo prime p, and a != 0.
57     int m = (int)sqrt((long double)(p - 1)) + 1;
58     unordered_map<long long, int> baby;
59     long long cur = 1;
60     for (int j = 0; j < m; j++) {

```

```

61         if (!baby.count(cur)) baby[cur] = j;
62         cur = (i128)cur * g % p;
63     }
64     long long step = mod_pow(mod_pow(g, m, p), p - 2, p);
65     cur = a % p;
66     for (int i = 0; i <= m; i++) {
67         auto it = baby.find(cur);
68         if (it != baby.end()) return 1LL * i * m + it->second;
69         cur = (i128)cur * step % p;
70     }
71     return -1;
72 }
73
74 vector<long long> kth_roots_mod_prime(long long a, long long k, long long p) {
75     a %= p;
76     if (a < 0) a += p;
77     if (a == 0) return {0};
78     if (p == 2) return {1};
79
80     long long g = primitive_root_prime(p);
81     long long ind = discrete_log_prime_root(g, a, p); // a = g^ind
82     if (ind == -1) return {};
83
84     long long mod = p - 1, d = gcd(k, mod);
85     if (ind % d) return {};
86
87     long long kk = ((k / d) % (mod / d) + (mod / d)) % (mod / d);
88     long long b = ind / d, md = mod / d;
89     long long y0 = (i128)b * inv_mod(kk, md) % md;
90
91     vector<long long> ans;
92     for (long long t = 0; t < d; t++) ans.push_back(mod_pow(g, y0 + t * md, p));
93     sort(ans.begin(), ans.end());
94     ans.erase(unique(ans.begin(), ans.end()), ans.end());
95     return ans;
96 }

```

5.15 FFT

Use: Polynomial convolution over doubles.

Complexity: $O(n \log n)$.

Pitfalls: Rounding fails for large coefficient energy; n rounded to power of two.

Assumptions/deps: Requires complex literals and helper macros `sz/all/rep`.

```

1 /**
2  * Description: fft(a) computes  $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot k x / N)$ 
3  * for all  $k$ .  $N$  must be a power of 2.
4  * Useful for convolution:

```

```

4  \texttt{conv}(a, b) = c\}, where  $c[x] = \sum a[i]b[x-i]$ .
5  For convolution of complex numbers or more than two vectors: FFT, multiply
6  pointwise, divide by  $n$ , reverse(start+1, end), FFT back.
7  Rounding is safe if  $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ 
8  (in practice  $10^{16}$ ; higher for random inputs).
9  Otherwise, use NTT/FFTMod.
10 * Time:  $O(N \log N)$  with  $N = |A|+|B|$  ( $\tilde{1s}$  for  $N=2^{22}$ )
11 * Status: somewhat tested
12 */
13 #pragma once
14
15 using ll = long long;
16 using vi = vector<int>;
17 #ifndef sz
18 #define sz(x) (int)(x).size()
19 #endif
20 #ifndef all
21 #define all(x) begin(x), end(x)
22 #endif
23 #ifndef rep
24 #define rep(i,a,b) for (int i = (a); i < (b); ++i)
25 #endif
26 using namespace std::complex_literals;
27
28 typedef complex<double> C;
29 typedef vector<double> vd;
30 void fft(vector<C>& a) {
31     int n = sz(a), L = 31 - __builtin_clz(n);
32     static vector<complex<long double>> R(2, 1);
33     static vector<C> rt(2, 1); // (~ 10% faster if double)
34     for (static int k = 2; k < n; k *= 2) {
35         R.resize(n); rt.resize(n);
36         auto x = polar(1.0L, acos(-1.0L) / k);
37         rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
38     }
39     vi rev(n);
40     rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
41     rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
42     for (int k = 1; k < n; k *= 2)
43         for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
44             // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled) /// include-line
45             auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k]; /// exclude-line
46             C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]); /// exclude-line
47             a[i + j + k] = a[i + j] - z;
48             a[i + j] += z;
49         }
50 }
51 vd conv(const vd& a, const vd& b) {
52     if (a.empty() || b.empty()) return {};
53     vd res(sz(a) + sz(b) - 1);
54     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
55     vector<C> in(n), out(n);

```

```

56     copy(all(a), begin(in));
57     rep(i,0,sz(b)) in[i].imag(b[i]);
58     fft(in);
59     for (C& x : in) x *= x;
60     rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
61     fft(out);
62     rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
63     return res;
64 }

```

5.16 FFT Mod

Use: Convolution modulo arbitrary int.

Complexity: $O(n \log n)$.

Pitfalls: Inputs must be in $[0, M)$; precision bound matters.

Assumptions/deps: Requires `fft_v2.cpp` before this snippet.

```

1  /**
2   * Description: Higher precision FFT, can be used for convolutions modulo arbitrary
3   * as long as  $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$  (in practice  $10^{16}$ 
4   * or higher).
5   * Inputs must be in  $[0, \text{mod})$ .
6   * Time:  $O(N \log N)$ , where  $N = |A|+|B|$  (twice as slow as NTT or FFT)
7   * Status: stress-tested
8   */
9  // Requires fft_v2.cpp printed above this snippet (defines C, fft, ll, rep, sz).
10 typedef vector<ll> vl;
11 template<int M> vl convMod(const vl &a, const vl &b) {
12     if (a.empty() || b.empty()) return {};
13     vl res(sz(a) + sz(b) - 1);
14     int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
15     vector<C> L(n), R(n), outs(n), outl(n);
16     rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
17     rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
18     fft(L), fft(R);
19     rep(i,0,n) {
20         int j = -i & (n - 1);
21         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
22         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
23     }
24     fft(outl), fft(outs);
25     rep(i,0,sz(res)) {
26         ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
27         ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
28         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
29     }
30     return res;
31 }

```

5.17 Fraction Finder

Use: Binary search over rationals with bounded numerator/denominator.

Complexity: $O(\log N)$ calls.

Pitfalls: Predicate must be monotone and true at hi; overflow possible in mediants.

Assumptions/deps: Uses ll.

```

1  /**
2   * Description: Given  $f$  and  $N$ , finds the smallest fraction  $p/q$  in  $[0, 1]$ 
3   * such that  $f(p/q)$  is true, and  $p, q \leq N$ .
4   * You may want to throw an exception from  $f$  if it finds an exact solution,
5   * in which case  $N$  can be removed.
6   * Usage: fracBS([f](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
7   * Time:  $O(\log(N))$ 
8   * Status: stress-tested for  $n \leq 300$ 
9   */
10
11 struct Frac { ll p, q; };
12
13 template<class F>
14 Frac fracBS(F f, ll N) {
15     bool dir = 1, A = 1, B = 1;
16     Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
17     if (f(lo)) return lo;
18     assert(f(hi));
19     while (A || B) {
20         ll adv = 0, step = 1; // move hi if dir, else lo
21         for (int si = 0; step; (step *= 2) >= si) {
22             adv += step;
23             Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
24             if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
25                 adv -= step; si = 2;
26             }
27         }
28         hi.p += lo.p * adv;
29         hi.q += lo.q * adv;
30         dir = !dir;
31         swap(lo, hi);
32         A = B; B = !adv;
33     }
34     return dir ? hi : lo;
35 }

```

5.18 Zeta and Mobius Transforms

Use: Subset/superset/divisor/multiple transforms.

Complexity: $O(n \log n)$ or harmonic prime loops.

Pitfalls: Subset size must be power of two. Divisor versions need prime enumeration.

Assumptions/deps: Depends on PrimeEnumerate(n) for divisor/multiple transforms.

```

1  #include<vector>
2  template<typename T>
3  void SubsetZetaTransform(vector<T>& v) {
4      const int n = v.size(); // n must be a power of 2
5      for (int j = 1; j < n; j <= 1) {
6          for (int i = 0; i < n; i++)
7              if (i & j) v[i] += v[i ^ j];
8      }
9  }
10
11 template<typename T>
12 void SubsetMobiusTransform(vector<T>& v) {
13     const int n = v.size(); // n must be a power of 2
14     for (int j = 1; j < n; j <= 1) {
15         for (int i = 0; i < n; i++)
16             if (i & j) v[i] -= v[i ^ j];
17     }
18 }
19
20 template<typename T>
21 void SupersetZetaTransform(vector<T>& v) {
22     const int n = v.size(); // n must be a power of 2
23     for (int j = 1; j < n; j <= 1) {
24         for (int i = 0; i < n; i++)
25             if (i & j) v[i ^ j] += v[i];
26     }
27 }
28
29 template<typename T>
30 void SupersetMobiusTransform(vector<T>& v) {
31     const int n = v.size(); // n must be a power of 2
32     for (int j = 1; j < n; j <= 1) {
33         for (int i = 0; i < n; i++)
34             if (i & j) v[i ^ j] -= v[i];
35     }
36 }
37
38 template<typename T>
39 void DivisorZetaTransform(vector<T>& v) {
40     const int n = (int)v.size() - 1;
41     for (int p : PrimeEnumerate(n)) {
42         for (int i = 1; i * p <= n; i++)

```

```

43     v[i * p] += v[i];
44 }
45 }
46
47 template<typename T>
48 void DivisorMobiusTransform(vector<T>& v) {
49     const int n = (int)v.size() - 1;
50     for(int p : PrimeEnumerate(n)) {
51         for (int i = n / p; i; i--)
52             v[i * p] -= v[i];
53     }
54 }
55
56 template<typename T>
57 void MultipleZetaTransform(vector<T>& v) {
58     const int n = (int)v.size() - 1;
59     for(int p : PrimeEnumerate(n)) {
60         for (int i = n / p; i; i--)
61             v[i] += v[i * p];
62     }
63 }
64
65 template<typename T>
66 void MultipleMobiusTransform(vector<T>& v) {
67     const int n = (int)v.size() - 1;
68     for (int p : PrimeEnumerate(n)) {
69         for (int i = 1; i * p <= n; i++)
70             v[i] -= v[i * p];
71     }
72 }

```

6 Geometry

Checklist: integer or floating? EPS? collinear kept or removed? boundary included? polygon order? coordinates up to 10^9 may need `__int128`. Circles need tangent/same-center cases.

6.1 Geometry Base

Use: Shared floating 2D point/vector primitives.

Complexity: Each primitive $O(1)$.

Pitfalls: Exact `operator<` but EPS equality; avoid mixing with integer exact logic.

Assumptions/deps: Copy before other floating geometry snippets.

```

1  /*
2  GEOMETRY CHECKLIST
3  1. int or floating?
4  2. Need EPS?
5  3. Coordinates up to 1e9 => integer cross may need __int128.
6  4. Segment intersection: endpoint? overlap? collinear?
7  5. Convex hull: keep or remove collinear points?
8  6. Polygon vertices clockwise or counterclockwise?
9  7. Area is signed; use abs(area2)/2.
10 8. For angle/sqrt, clamp values if needed.
119. For circles, handle tangent and same-center cases.
12 */
13
14 // Shared floating-point geometry base.
15 // Source: KACTL Point style, simplified for notebook use.
16 // Complexity: each primitive is O(1).
17 // Pitfalls: operator< is exact for sort/set; geometric equality uses EPS.
18 // Usage: copy this before other floating geometry snippets.
19 using T = long double;
20 const T EPS = 1e-12;
21
22 int sgn(T x) { return (x > EPS) - (x < -EPS); }
23
24 struct P {
25     T x, y;
26     bool operator<(P p) const {
27         if (x != p.x) return x < p.x;
28         return y < p.y;
29     }
30     bool operator==(P p) const {
31         return sgn(x - p.x) == 0 && sgn(y - p.y) == 0;
32     }
33     P operator+(P p) const { return {x + p.x, y + p.y}; }
34     P operator-(P p) const { return {x - p.x, y - p.y}; }
35     P operator*(T k) const { return {x * k, y * k}; }
36     P operator/(T k) const { return {x / k, y / k}; }
37 };
38
39 T dot(P a, P b) { return a.x * b.x + a.y * b.y; }
40 T cross(P a, P b) { return a.x * b.y - a.y * b.x; }
41 T cross(P a, P b, P c) { return cross(b - a, c - a); }
42 T norm2(P a) { return dot(a, a); }
43 T norm(P a) { return sqrt((double)norm2(a)); }
44 P perp(P a) { return {-a.y, a.x}; }
45
46 int ori(P a, P b, P c) { return sgn(cross(a, b, c)); }
47
48 bool onSegment(P a, P b, P p) {
49     return ori(a, b, p) == 0 && sgn(dot(p - a, p - b)) <= 0;
50 }

```

6.2 Integer Geometry

Use: Exact orientation for large integer coordinates.

Complexity: $O(1)$.

Pitfalls: Use `__int128` for products near 10^{18} ; return only sign.

Assumptions/deps: Defines `IP`, `i128`.

```
1 // Integer geometry orientation helper for large coordinates.
2 // Complexity:  $O(1)$ .
3 // Pitfalls: use __int128 for cross when coordinates/products may overflow long long.
4 // Usage: int o = ori_i(a, b, c); // -1, 0, +1
5 using ll = long long;
6 __extension__ typedef __int128 i128;
7
8 struct IP {
9     ll x, y;
10 };
11
12 i128 cross_i(IP a, IP b, IP c) {
13     return (i128)(b.x - a.x) * (c.y - a.y) -
14            (i128)(b.y - a.y) * (c.x - a.x);
15 }
16
17 int ori_i(IP a, IP b, IP c) {
18     i128 v = cross_i(a, b, c);
19     return (v > 0) - (v < 0);
20 }
```

6.3 Convex Hull and Geometry Routines

Use: Segments, polygon tests, hull, diameter, circle intersections.

Complexity: Most $O(1)$; polygon/hull $O(n)$ or $O(n \log n)$.

Pitfalls: Hull removes collinear boundary points with $\leq$. Line intersection assumes nonparallel lines.

Assumptions/deps: Depends on `geometry_base.cpp`; hull output has no duplicate last point.

```
1 // Core 2D geometry routines. Depends on geometry_base.cpp.
2 // Sources: KACTL OnSegment/LineIntersection/InsidePolygon/ConvexHull/HullDiameter/
3 //           CircleLine/CircleIntersection style.
4 // Complexity: listed per function below. Uses 0-indexed vectors.
5 // Usage: copy geometry_base.cpp first.
6
7 // segment intersection.  $O(1)$ .
8 // Pitfalls: segInter only says yes/no; overlapping intersection needs separate
9 //           handling.
10 bool segInter(P a, P b, P c, P d) {
```

```
9     int oa = ori(c, d, a), ob = ori(c, d, b);
10    int oc = ori(a, b, c), od = ori(a, b, d);
11    if (oa == 0 && onSegment(c, d, a)) return true;
12    if (ob == 0 && onSegment(c, d, b)) return true;
13    if (oc == 0 && onSegment(a, b, c)) return true;
14    if (od == 0 && onSegment(a, b, d)) return true;
15    return oa * ob < 0 && oc * od < 0;
16 }
17
18 // line intersection/projection/distances.  $O(1)$ .
19 // Pitfalls: lineInter assumes non-parallel non-zero-length lines.
20 P lineInter(P a, P b, P c, P d) {
21     P r = b - a, s = d - c;
22     return a + r * (cross(c - a, s) / cross(r, s));
23 }
24
25 P projLine(P a, P b, P p) {
26     P ab = b - a;
27     return a + ab * (dot(p - a, ab) / norm2(ab));
28 }
29
30 T distLine(P a, P b, P p) {
31     return abs(cross(b - a, p - a)) / norm(b - a);
32 }
33
34 T distSegment(P a, P b, P p) {
35     if (a == b) return norm(p - a);
36     if (sgn(dot(p - a, b - a)) <= 0) return norm(p - a);
37     if (sgn(dot(p - b, a - b)) <= 0) return norm(p - b);
38     return distLine(a, b, p);
39 }
40
41 // polygon signed double-area.  $O(n)$ .
42 // Pitfalls: clockwise order gives negative area.
43 T area2(const vector<P>& p) {
44     T s = 0;
45     for (int i = 0, n = (int)p.size(); i < n; i++) s += cross(p[i], p[(i + 1) % n]);
46     return s;
47 }
48
49 // point in polygon: 0 outside, 1 inside, 2 boundary.  $O(n)$ .
50 // Pitfalls: works for simple polygons; boundary is checked first.
51 int inPoly(const vector<P>& poly, P q) {
52     bool in = false;
53     int n = (int)poly.size();
54     for (int i = 0, j = n - 1; i < n; j = i++) {
55         P a = poly[i], b = poly[j];
56         if (onSegment(a, b, q)) return 2;
57         bool cond = (a.y > q.y) != (b.y > q.y);
58         if (cond) {
59             T x = a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
60             if (sgn(x - q.x) > 0) in = !in;
61         }
62     }
63     return in;
64 }
```



```

61     }
62 }
63 return in;
64 }
65
66 // convex hull in CCW order. O(n log n).
67 // Pitfalls: <= removes collinear boundary points; change to < to keep them.
68 vector<P> convexHull(vector<P> p) {
69     sort(p.begin(), p.end());
70     p.erase(unique(p.begin(), p.end(), p.end()));
71     if ((int)p.size() <= 1) return p;
72     vector<P> h;
73     for (P pt : p) {
74         while ((int)h.size() >= 2 && sgn(cross(h[(int)h.size() - 2], h.back(), pt))
75             <= 0)
76             h.pop_back();
77         h.push_back(pt);
78     }
79     int lower = (int)h.size();
80     for (int i = (int)p.size() - 2; i >= 0; i--) {
81         P pt = p[i];
82         while ((int)h.size() > lower && sgn(cross(h[(int)h.size() - 2], h.back(), pt))
83             <= 0)
84             h.pop_back();
85         h.push_back(pt);
86     }
87     h.pop_back();
88     return h;
89 }
90
91 // squared diameter of a convex polygon in CCW order. O(n).
92 // Pitfalls: run convexHull first; hull should have no duplicate last point.
93 T diameter2(const vector<P>& p) {
94     int n = (int)p.size();
95     if (n <= 1) return 0;
96     if (n == 2) return norm2(p[0] - p[1]);
97     T ans = 0;
98     int j = 1;
99     for (int i = 0; i < n; i++) {
100         int ni = (i + 1) % n;
101         while (abs(cross(p[ni] - p[i], p[(j + 1) % n] - p[i])) >
102             abs(cross(p[ni] - p[i], p[j] - p[i])))
103             j = (j + 1) % n;
104         ans = max(ans, norm2(p[i] - p[j]));
105     }
106     return ans;
107 }
108
109 // circle-line intersection. O(1).
110 // Pitfalls: a and b must be distinct; returns 0/1/2 points.
111 vector<P> circleLine(P c, T r, P a, P b) {
112     P ab = b - a;

```

```

111     P h = a + ab * (dot(c - a, ab) / norm2(ab));
112     T d2 = norm2(h - c);
113     if (sgn(d2 - r * r) > 0) return {};
114     P dir = ab / norm(ab);
115     if (sgn(d2 - r * r) == 0) return {h};
116     T len = sqrt((double)max((T)0, r * r - d2));
117     return {h - dir * len, h + dir * len};
118 }
119
120 // circle-circle intersection. O(1).
121 // Pitfalls: same center returns empty for both infinite and no intersections.
122 vector<P> circleCircle(P a, T r, P b, T R) {
123     P ab = b - a;
124     T d2 = norm2(ab), d = sqrt((double)d2);
125     if (sgn(d) == 0) return {};
126     T x = (d2 + r * r - R * R) / (2 * d);
127     T h2 = r * r - x * x;
128     if (sgn(h2) < 0) return {};
129     P v = ab / d, p = a + v * x, q = perp(v);
130     if (sgn(h2) == 0) return {p};
131     T h = sqrt((double)max((T)0, h2));
132     return {p - q * h, p + q * h};
133 }

```

6.4 Point in Convex Polygon

Use: Fast containment query in a convex polygon.

Complexity: $O(\log n)$ per query.

Pitfalls: Polygon must be CCW and no duplicate last point; strict excludes boundary.

Assumptions/deps: Depends on geometry_base.cpp.

```

1 // Point in convex polygon in O(log n). Depends on geometry base.
2 // Source: KACTL PointInsideHull style.
3 // Complexity: O(log n) per query after CCW convex hull is built.
4 // Pitfalls: polygon must be CCW and have no duplicate last point; strict excludes
5 // boundary.
6 // Usage: bool ok = inConvex(poly, q, false);
7 bool inConvex(const vector<P>& p, P q, bool strict = false) {
8     int n = (int)p.size();
9     if (n == 0) return false;
10    if (n == 1) return !strict && p[0] == q;
11    if (n == 2) return !strict && onSegment(p[0], p[1], q);
12    int a = ori(p[0], p[1], q), b = ori(p[0], p[n - 1], q);
13    if (a < 0 || b > 0) return false;
14    if (strict && (a == 0 || b == 0)) return false;
15    int l = 1, r = n - 1;
16    while (r - l > 1) {

```

```

16     int m = (l + r) / 2;
17     if (ori(p[0], p[m], q) >= 0) l = m;
18     else r = m;
19 }
20 int o = ori(p[l], p[(l + 1) % n], q);
21 return strict ? o > 0 : o >= 0;
22 }

```

6.5 Closest Pair

Use: Minimum squared distance among points.

Complexity: $O(n \log n)$.

Pitfalls: Returns squared distance; needs at least 2 points; duplicates return 0.

Assumptions/deps: Depends on `geometry_base.cpp`.

```

1 // Closest pair of points. Depends on geometry base.
2 // Source: standard divide-and-conquer closest pair.
3 // Complexity:  $O(n \log n)$ .
4 // Pitfalls: returns squared distance; duplicate points give 0; needs  $n \geq 2$ .
5 // Usage: auto [d2, pr] = closestPair(points);
6 pair<T, pair<P, P>> closestPair(vector<P> p) {
7     assert((int)p.size() >= 2);
8     sort(p.begin(), p.end(), [](P a, P b) {
9         if (a.x != b.x) return a.x < b.x;
10        return a.y < b.y;
11    });
12     vector<P> tmp(p.size());
13     pair<T, pair<P, P>> ans = {norm2(p[0] - p[1]), {p[0], p[1]}};
14
15     auto upd = [&](P a, P b) {
16         T d = norm2(a - b);
17         if (d < ans.first) ans = {d, {a, b}};
18     };
19
20     function<void(int, int)> solve = [&](int l, int r) {
21         if (r - l <= 3) {
22             for (int i = l; i < r; i++)
23                 for (int j = i + 1; j < r; j++) upd(p[i], p[j]);
24             sort(p.begin() + l, p.begin() + r, [](P a, P b) { return a.y < b.y; });
25             return;
26         }
27         int m = (l + r) / 2;
28         T mx = p[m].x;
29         solve(l, m);
30         solve(m, r);
31         merge(p.begin() + l, p.begin() + m, p.begin() + m, p.begin() + r,
32              tmp.begin(), [](P a, P b) { return a.y < b.y; });
33         copy(tmp.begin(), tmp.begin() + (r - l), p.begin() + l);

```

```

34     int sz = 0;
35     T d = sqrt((double)ans.first);
36     for (int i = l; i < r; i++) if (fabs1(p[i].x - mx) <= d + EPS) {
37         for (int j = sz - 1; j >= 0 && p[i].y - tmp[j].y <= d + EPS; j--)
38             upd(p[i], tmp[j]);
39         tmp[sz++] = p[i];
40     }
41 }
42 };
43 solve(0, (int)p.size());
44 return ans;
45 }

```

6.6 Circle Tangents

Use: External/internal tangent points between two circles or point-circle.

Complexity: $O(1)$.

Pitfalls: Use $r2=0$ for point tangents; negate radius for internal tangents. Same center returns empty.

Assumptions/deps: Depends on `geometry_base.cpp`.

```

1 // Tangent points between two circles. Depends on geometry base.
2 // Source: KACTL CircleTangents style.
3 // Complexity:  $O(1)$ .
4 // Pitfalls: use  $r2 = 0$  for tangents from point  $c2$  to circle  $c1$ ; negate  $r2$  for
5 // internal tangents.
6 // Usage: auto t = circleTangents(c, r, p, 0); // pairs {on circle, p}
7 vector<pair<P, P>> circleTangents(P c1, T r1, P c2, T r2) {
8     P d = c2 - c1;
9     T dr = r1 - r2, d2 = norm2(d), h2 = d2 - dr * dr;
10    if (sgn(d2) == 0 || sgn(h2) < 0) return {};
11    vector<pair<P, P>> out;
12    for (T sign : {-1.0L, 1.0L}) {
13        P v = (d * dr + perp(d) * sqrt((double)max((T)0, h2)) * sign) / d2;
14        out.push_back({c1 + v * r1, c2 + v * r2});
15    }
16    if (sgn(h2) == 0) out.pop_back();
17    return out;

```

6.7 Minimum Enclosing Circle

Use: Smallest circle covering all points.

Complexity: Expected $O(n)$.

Pitfalls: Randomized; circumcenter fails for collinear triples but algorithm should call only when needed. EPS matters.

Assumptions/deps: Depends on geometry_base.cpp; uses shuffle, mt19937.

```

1 // Minimum enclosing circle for points in the plane.
2 // Source: KACTL MinEnclosingCircle, adapted to the shared geometry P.
3 // Complexity: expected  $O(n)$ .
4 // Pitfalls: randomized order; copy geometry base first; use EPS when checking
   containment.
5 // Usage: auto [c, r] = minimum_enclosing_circle(points);
6 P circumcenter(P a, P b, P c) {
7     b = b - a, c = c - a;
8     T d = 2 * cross(b, c);
9     return a + P{
10         (c.y * (b.x * b.x + b.y * b.y) - b.y * (c.x * c.x + c.y * c.y)) / d,
11         (b.x * (c.x * c.x + c.y * c.y) - c.x * (b.x * b.x + b.y * b.y)) / d
12     };
13 }
14
15 pair<P, T> minimum_enclosing_circle(vector<P> p) {
16     if (p.empty()) return {{0, 0}, 0};
17     shuffle(p.begin(), p.end(), mt19937(chrono::steady_clock::now().time_since_epoch
   ().count()));
18     P c = p[0];
19     T r = 0, mult = 1 + 1e-10;
20     for (int i = 0; i < (int)p.size(); i++) if (norm(c - p[i]) > r * mult) {
21         c = p[i], r = 0;
22         for (int j = 0; j < i; j++) if (norm(c - p[j]) > r * mult) {
23             c = (p[i] + p[j]) / 2;
24             r = norm(c - p[i]);
25             for (int k = 0; k < j; k++) if (norm(c - p[k]) > r * mult) {
26                 c = circumcenter(p[i], p[j], p[k]);
27                 r = norm(c - p[i]);
28             }
29         }
30     }
31     return {c, r};
32 }

```

7 Dynamic Programming

7.1 Knuth Optimization

Use: Interval DP with monotone split bounds.

Complexity: $O(n^2)$ after prefix/cost setup.

Pitfalls: Only valid when Knuth quadrangle/monotone conditions hold. Snippet is 1-indexed and uses #define int long long.

Assumptions/deps: Standalone example with main; globals sized by N.

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define F first
5 #define S second
6 #define all(x) (x).begin(), (x).end()
7 #define log(x) cout << #x << " = " << x << " "
8 #define logl(x) cout << #x << " = " << x << endl
9 #define print(a) cout << #a << " = "; \
10 for(auto x: a) cout << x << " "; cout << endl;
11 template<typename T, typename U>
12 ostream &operator<<(ostream &os, const pair<T, U> &p)
13 {
14     return os << "(" << p.F << ", " << p.S << ")";
15 }
16 const int N = 5e3 + 5;
17 int a[N], pref[N];
18 int dp[N][N], opt[N][N];
19 int C(int l, int r)
20 {
21     return pref[r] - pref[l - 1];
22 }
23 int32_t main()
24 {
25     ios::sync_with_stdio(0); cin.tie(NULL);
26     int n; cin >> n;
27     for(int i = 1; i <= n; i++)
28     {
29         cin >> a[i];
30         pref[i] = pref[i - 1] + a[i];
31     }
32     for(int i = 1; i <= n; i++)
33         opt[i][i] = i;
34
35     for(int len = 2; len <= n; len++)
36     {
37         for(int i = 1; i + len - 1 <= n; i++)
38         {
39             int j = i + len - 1;
40             int lo = opt[i][j - 1];
41             int hi = opt[i + 1][j];
42             pair<int, int> best = {1e18, -1};
43             for(int k = max(i, lo); k <= min(hi, j - 1); k++)
44             {
45                 best = min(best, make_pair(dp[i][k] + dp[k + 1][j], k));
46             }
47             dp[i][j] = best.first + C(i, j);
48             opt[i][j] = best.second;

```

```

49     assert(opt[i][j] != -1);
50 }
51 }
52 cout << dp[1][n] << "\n";

```

7.2 Divide and Conquer DP

Use: Optimize DP with monotone optimal split.

Complexity: Typically $O(mn \log n)$ here; tighter variants can be $O(mn)$.

Pitfalls: Do not use unless monotonicity of `opt` is proven/tested. Cost $C(k, j)$ must be fast.

Assumptions/deps: Implement `C`; recurrence assumes previous row and 0-indexed intervals.

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define F first
5 #define S second
6 #define all(x) (x).begin(), (x).end()
7 #define log(x) cout << #x << " = " << x << " "
8 #define logl(x) cout << #x << " = " << x << endl
9 #define print(a) cout << #a << " = "; \
10 for(auto x: a) cout << x << " "; cout << endl;
11 template<typename T, typename U>
12 ostream &operator<<(ostream &os, const pair<T, U> &p)
13 {
14     return os << "(" << p.F << ", " << p.S << ")";
15 }
16 const int N = 3e5 + 5;
17 int n, m;
18 vector<int> dp_bef, dp_cur;
19 int a[N], pref[N];
20 int cost(int l, int r)
21 {
22     int sum = pref[r] - pref[l - 1];
23     return sum * sum;
24 }
25 void comp(int l, int r, int lo, int hi)
26 {
27     if(l > r) return;
28     int mid = (l + r) / 2;
29     pair<int, int> best = {(int)1e18, -1};
30     for(int k = lo; k <= min(hi, mid); k++)
31     {
32         best = min(best, make_pair((k ? dp_bef[k - 1] : 0) + cost(k, mid), k));
33     }
34     //log(l); log(r); log(mid); logl(best);

```

```

35     dp_cur[mid] = best.first;
36     int opt = best.second;
37     assert(opt != -1);
38     comp(l, mid - 1, lo, opt);
39     comp(mid + 1, r, opt, hi);
40 }
41 int32_t main()
42 {
43     ios::sync_with_stdio(0); cin.tie(NULL);
44     cin >> n >> m;
45     for(int i = 1; i <= n; i++)
46     {
47         cin >> a[i];
48         pref[i] = pref[i - 1] + a[i];
49     }
50     dp_bef.assign(n + 1, 0);
51     dp_cur.assign(n + 1, 0);
52     for(int j = 1; j <= n; j++) dp_cur[j] = cost(1, j);
53     dp_bef = dp_cur;
54     //print(dp_bef);
55     for(int i = 2; i <= m; i++)
56     {
57         comp(1, n, 1, n);
58         dp_bef = dp_cur;
59         //print(dp_cur);
60     }
61     cout << dp_cur[n] << '\n';
62 }

```

7.3 Divide and Conquer DP Notes

Use: Proof checklist for monotone DP optimization.

Complexity: Reference notes.

Pitfalls: The note is a guide, not a proof for a specific problem.

Assumptions/deps: Read before using the template.

```

1 Divide and Conquer is a dynamic programming optimization.
2
3 ## Preconditions
4
5 Some dynamic programming problems have a recurrence of the form:
6
7 $$
8 dp(i, j) = \min_{0 \leq k \leq j}
9 \left\{ dp(i - 1, k - 1) + C(k, j) \right\}
10 $$
11
12 where $ C(k, j) $ is a cost function and

```

```

13 $ dp(i, j) = 0 $ when $ j < 0 $.
14
15 Assume:
16 $$
17 0 \leq i < m, \quad 0 \leq j < n
18 $$
19 and evaluating $ C $ takes $ O(1) $ time.
20
21 A straightforward evaluation runs in:
22 $$
23 O(m n^2)
24 $$
25 There are $ m \times n $ states and $ n $ transitions per state.
26
27 Let $ opt(i, j) $ be the value of $ k $ that minimizes the recurrence.
28
29 If the cost function satisfies the quadrangle inequality, then we can prove:
30
31 $$
32 opt(i, j) \leq opt(i, j+1)
33 $$
34
35 This is called the monotonicity condition.
36 For fixed $ i $, the optimal splitting point increases as $ j $ increases.
37
38 This allows us to reduce the search range when computing transitions.
39
40 If we know $ opt(i, j) $, then for any $ j' < j $:
41
42 $$
43 opt(i, j') \leq opt(i, j)
44 $$
45
46 So when computing $ opt(i, j') $, we only need to search a smaller interval.
47
48 ## Divide and Conquer Optimization
49
50 We apply a divide and conquer strategy:
51
52 1. Compute $ opt(i, \lfloor n/2 \rfloor) $.
53 2. Recursively compute:
54     - Left half using upper bound $ opt(i, n/2) $
55     - Right half using lower bound $ opt(i, n/2) $
56
57 By maintaining valid bounds on $ opt $, we reduce the total complexity to:
58
59 $$
60 O(m n \log n)
61 $$
62
63 Each possible value of $ opt(i, j) $ appears in at most $ \log n $ recursion levels.
64

```

```

35 Importantly, the method does not depend on how balanced the splits are.
36 Across each recursion level, every candidate $ k $ is used at most twice.
37
38 ## Generic Implementation
39
40 Below is a common template.
41 The function 'compute' calculates one row $ i $ of DP values ('dp_cur')
42 using the previous row ('dp_before').
43
44 It must be called as:
45
46 $$
47 'compute(0, n-1, 0, n-1)'
48 $$
49
50 The function 'solve' computes all $ m $ rows.
51
52
53
54 ## Things to Watch Out For
55
56 The hardest part of Divide and Conquer DP is proving the monotonicity of $ opt $.
57
58 A common sufficient condition is the quadrangle inequality:
59
60 $$
61 C(a, c) + C(b, d) \leq C(a, d) + C(b, c)
62 \quad \text{for all } a \leq b \leq c \leq d
63 $$
64
65 Many Divide and Conquer DP problems can also be solved using the
66 Convex Hull Trick, and vice versa.
67 Understanding both techniques is highly recommended.

```

8 Misc

8.1 Quick Select

Use: Get k th element / partition order statistic.

Complexity: Average/guaranteed linear for `nth_element`.

Pitfalls: After call, only k th is fixed; sides are partitioned, not sorted.

Assumptions/deps: k is 0-indexed iterator offset.

```

1 nth_element(a.begin(), a.begin()+k, a.end());

```

8.2 mt19937 RNG

Use: Random 32-bit generator.

Complexity: $O(1)$ per value.

Pitfalls: Modulo causes bias; use distributions if exact uniformity matters.

Assumptions/deps: Includes full small demo with main.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
5
6 int main() {
7     int x = rng(); // random 32-bit integer
8     int y = rng() % 100; // 0 to 99
9     cout << x << " " << y << "\n";
10 }
```

8.3 mt19937_64 RNG

Use: Random 64-bit generator.

Complexity: $O(1)$ per value.

Pitfalls: Seed once; do not reseed inside tight loops.

Assumptions/deps: Requires chrono, mt19937_64.

```
1 mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());
2 unsigned long long x = rng64();
```

8.4 Random Hashing

Use: Assign random xor keys to values.

Complexity: Expected $O(1)$ map operations.

Pitfalls: Hash is probabilistic. Requires stable rng and clearing map when state universe changes.

Assumptions/deps: Depends on rng; uint64_t.

```
1 uint64_t rnd() {
2     return rng();
3 }
4 unordered_map<int, uint64_t> h;
5 uint64_t getHash(int x) {
6     if (!h.count(x)) h[x] = rnd();
7     return h[x];
8 }
```

```
9 cur ^= getHash(x); // toggle x
```

8.5 Zobrist Hashing

Use: Incremental xor hash for states.

Complexity: Build $O(positions \cdot values)$, update/hash $O(1)$ or $O(n)$.

Pitfalls: Probabilistic collisions; state values must be valid indices.

Assumptions/deps: Uses mt19937_64; uint64_t.

```
1 // Zobrist hashing: xor random keys for incremental state hashes.
2 // Source: standard Zobrist/tabulation hashing.
3 // Complexity: build  $O(positions * values)$ , update/query  $O(1)$ .
4 // Pitfalls: probabilistic collisions; state values must be in range.
5 // Usage: h ^= key[pos][old]; h ^= key[pos][now];
6 struct Zobrist {
7     vector<vector<uint64_t>> key;
8
9     Zobrist(int positions, int values, uint64_t seed = chrono::steady_clock::now().
10         time_since_epoch().count()) {
11         mt19937_64 rng(seed);
12         key.assign(positions, vector<uint64_t>(values));
13         for (auto& row : key)
14             for (uint64_t& x : row) x = rng();
15     }
16
17     uint64_t hash(const vector<int>& state) const {
18         uint64_t h = 0;
19         for (int i = 0; i < (int)state.size(); i++)
20             if (state[i] >= 0) h ^= key[i][state[i]];
21         return h;
22     }
23 };
```

8.6 GNU Extensions

Use: PBDS ordered sets/maps and rope snippets.

Complexity: $O(\log n)$ usual PBDS/rope ops.

Pitfalls: Examples are not one compilable file. PBDS multiset with less_equal is risky; pair trick is safer.

Assumptions/deps: GNU C++ only.

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 // NOTE: examples below are not one compilable file; paste only the needed block.
```

```

4 using namespace __gnu_pbds;
5
6 template<typename T>
7 using ordered_set = tree<
8     T,
9     null_type,
10    less<T>,
11    rb_tree_tag,
12    tree_order_statistics_node_update>;
13
14 template<typename K, typename V>
15 using ordered_map = tree<
16     K,
17     V,
18     less<K>,
19     rb_tree_tag,
20     tree_order_statistics_node_update>;
21
22 ordered_set<int> s;
23
24 s.insert(x);
25 s.erase(x);
26
27 // number of elements strictly smaller than x
28 int k = s.order_of_key(x);
29
30 // iterator to k-th (0-indexed) element
31 auto it = s.find_by_order(k);
32
33 using pii = pair<int,int>;
34
35 ordered_set<pii> ms;
36 int id = 0;
37
38 ms.insert({value, id++});
39
40 // erase one occurrence
41 ms.erase(ms.lower_bound({value, 0}));
42
43 tree<int, null_type, less_equal<int>,
44     rb_tree_tag,
45     tree_order_statistics_node_update> ms;
46
47 #include <ext/rope>
48 using namespace __gnu_cxx;
49
50 rope<char> r;
51
52 r.insert(pos, "abc");
53 r.erase(pos, len);
54 r.replace(pos, len, "xyz");
55

```

```

56 char c = r[i];

```

8.7 GNU Extensions Notes

Use: Short PBDS/rope reminders.

Complexity: Reference notes.

Pitfalls: Do not paste empty note blocks; use code snippet above.

Assumptions/deps: GNU C++ only.

```

1 ## Ordered Set / Ordered Map (PBDS)
2
3 Header:
4
5
6 **Ordered Set**
7
8
9
10 **Ordered Map**
11
12
13
14 **Supported operations**
15
16
17
18 Time complexity:  $O(\log n)$ 
19
20 ---
21
22 ## Ordered Multiset
23
24 PBDS does not support duplicates directly.
25 Trick: store pairs.
26
27
28
29 Alternative trick (less clean):
30
31
32 ---
33
34 ## Rope
35
36 Header:
37
38
39 Rope is useful for fast string modifications.

```

```

40
41
42
43 Time complexity:
44 $$
45 \text{insert / erase / substring} = O(\log n)
46 $$
47
48 Common use cases:
49
50 - Editor simulation
51 - Large string modifications
52 - Persistent-like behavior

```

8.8 Simulated Annealing Skeleton

Use: Approximate optimization when exact method is hard.

Complexity: Heuristic.

Pitfalls: Risky in ICPC: tune step, time limit, random direction, and score. No correctness guarantee.

Assumptions/deps: Depends on geometry point P and user functions.

```

1 // find approximate point minimizing max distance
2 // find point with best score
3 // optimize non-convex function
4 P cur = start;
5 double step = initial_step;
6
7 while (step > 1e-9) {
8     P nxt = cur + random_direction() * step;
9     if (score(nxt) < score(cur))
10         cur = nxt;
11     step *= 0.98;
12 }

```

8.9 Debug Checklist

Use: Contest debugging prompts for WA/RE/TLE/MLE.

Complexity: Checklist.

Pitfalls: Use before rewriting blindly; check output format and resets.

Assumptions/deps: Plain text notes.

```

1 Pre-submit:
2 Write a few simple test cases if sample is not enough.

```

```

3 Are time limits close? If so, generate max cases.
4 Is the memory usage fine?
5 Could anything overflow?
6 Make sure to submit the right file.
7
8 Wrong answer:
9 Print your solution! Print debug output, as well.
10 Are you clearing all data structures between test cases?
11 Can your algorithm handle the whole range of input?
12 Read the full problem statement again.
13 Do you handle all corner cases correctly?
14 Have you understood the problem correctly?
15 Any uninitialized variables?
16 Any overflows?
17 Confusing N and M, i and j, etc.?
18 Are you sure your algorithm works?
19 What special cases have you not thought of?
20 Are you sure the STL functions you use work as you think?
21 Add some assertions, maybe resubmit.
22 Create some testcases to run your algorithm on.
23 Go through the algorithm for a simple case.
24 Go through this list again.
25 Explain your algorithm to a teammate.
26 Ask the teammate to look at your code.
27 Go for a small walk, e.g. to the toilet.
28 Is your output format correct? (including whitespace)
29 Rewrite your solution from the start or let a teammate do it.
30
31 Runtime error:
32 Have you tested all corner cases locally?
33 Any uninitialized variables?
34 Are you reading or writing outside the range of any vector?
35 Any assertions that might fail?
36 Any possible division by 0? (mod 0 for example)
37 Any possible infinite recursion?
38 Invalidated pointers or iterators?
39 Are you using too much memory?
40 Debug with resubmits (e.g. remapped signals, see Various).
41
42 Time limit exceeded:
43 Do you have any possible infinite loops?
44 What is the complexity of your algorithm?
45 Are you copying a lot of unnecessary data? (References)
46 How big is the input and output? (consider scanf)
47 Avoid vector, map. (use arrays/unordered_map)
48 What do your teammates think about your algorithm?
49
50 Memory limit exceeded:
51 What is the max amount of memory your algorithm should need?
52 Are you clearing all data structures between test cases?

```